SHENKAR - ENGINEERING. DESIGN. ART.
The Pernick Faculty of Engineering

CalEyeZ

A Sensor-Fusion System for Automated Nutritional Analysis

Electrical & Electronic Engineering B.Sc. Final Project

Authors: Raz Dvora
Roi Tzur
Supervisors: Dr. Gabriela Dorfman Furman
Dr. Zeev Weissman

Capstone Project 2026

Acknowledgements

We would like to thank our academic supervisors, Dr. Gabriela Dorfman Furman and Dr. Zeev Weissman, for their guidance throughout this project. We are grateful for their patience as the project evolved - including the engineering decision to pivot the weight-acquisition subsystem from optical character recognition to a reverse-engineered Bluetooth Low Energy link - and for their feedback that pushed us toward a rigorous, evidence-based evaluation of our system.

We also thank our families for their unwavering support.

Table of Contents

1. Introduction

1.1. Project Overview

Project Title: CalEyeZ - Automated Sensor-Fusion Nutritional Analysis.

Problem Statement. Tracking nutrition manually is tedious and error-prone, which causes most users to abandon the habit. Two distinct engineering obstacles make **automation** hard. First, the **"What?"** problem: a single ("monolithic") neural network trained on a very large, mixed set of food classes becomes unstable, and teaching it new local dishes causes it to forget previously learned foods (*catastrophic forgetting*[1]). Second, the **"How much?"** problem: estimating food **weight** from a 2-D photograph is unreliable, because converting an estimated volume to a mass depends on food density, which varies wildly (a fluffy bun versus a dense steak of the same size).

Our Solution. CalEyeZ is a desktop edge-computing system that solves both problems with a **sensor-fusion** architecture:

- **"What?"** is solved by a **Smart Ensemble**: two specialised classifiers - a **Global** model (132 international food classes) and a **Local** model (Israeli cuisine) - managed by an **XGBoost Router** that inspects each image and forwards it to the expert most likely to be correct. This sidesteps catastrophic forgetting: new cuisines are added as a new expert, never by retraining the Global model.
- **"How much?"** is solved deterministically by reading the **exact weight** from a digital scale over a **Bluetooth Low Energy (BLE)** link, instead of guessing volume from pixels.

The identified food label and the measured weight are then fused and cross-referenced against the **USDA FoodData Central** database to produce a calorie and macronutrient breakdown.

Significance & Impact. The project demonstrates a practical electrical-engineering result: a **team of specialised models managed by a learned router** outperforms a single general model on a real, heterogeneous dataset, while remaining **modular and scalable**. Replacing pixel-based volume estimation with a direct gravimetric measurement turns the device from a "guessing tool" into a precise nutritional instrument.

Innovation & Novelty.

- A **learned arbiter** (XGBoost) that routes between deep-learning experts using their own uncertainty signals (confidence, entropy, margin) plus an **open-set "background" signal**, rather than a hand-tuned confidence threshold.
- **Sensor fusion** of a visual classifier with a reverse-engineered BLE scale (GATT serial profile), giving laboratory-grade weight with consumer-grade ease of use.
- A **torch-free ONNX edge build** that runs the full pipeline on a CPU-only device.

Target Audience. Health-conscious end users and dieters; dietitians/nutritionists who need objective image-based logging; and system/data engineers who need to extend the food vocabulary without destabilising the existing models.

[1] A classic failure mode of neural networks. Modern mitigations such as elastic weight consolidation (J. Kirkpatrick et al., "Overcoming catastrophic forgetting in neural networks," *PNAS*, vol. 114, no. 13, pp. 3521-3526, 2017, arXiv:1612.00796) trade plasticity against stability; CalEyeZ instead sidesteps the stability-plasticity dilemma entirely by isolating each domain in its own frozen expert.

Project Goals (summary of Section 1.3). Achieve $\geq 80\%$ validated system top-1 accuracy across the combined class vocabulary; validate the router-ensemble architecture against catastrophic forgetting; obtain precise weight via a deterministic hardware link; and deliver an end-to-end automated pipeline from a single image to a nutrition report.

Methodology. Data collection and cleaning (leakage-free splits) → transfer-learning of two YOLO11 classifiers → feature extraction and training of the XGBoost router → reverse-engineering of the BLE scale protocol → integration into a CustomTkinter desktop application → field validation on real photographs → export to an ONNX edge build.

1.2. Problem Definition

Problem Statement. Build a fully automated system that, from a single image of a meal on a scale, returns **what** the food is and **how much** of it there is, and converts that into an accurate nutritional report - without relying on the user to search a database or type in a weight.

1.2.1. Target Audience

- **End users / dieters** - want effortless, accurate logging, including local dishes that generic apps miss (e.g. jachnun, sabich, bourekas).
- **Dietitians / nutritionists** - want objective, image-based intake data instead of subjective self-reporting.
- **System / data engineers** - want to add new food classes without retraining or degrading the existing models.

1.2.2. Existing Solutions

- **MyFitnessPal (manual logging).** Large crowd-sourced database, but depends on the user searching for items and **typing in the weight**. High friction; accuracy is bounded by human estimation.
- **FoodVisor (visual volume estimation).** Uses deep learning to segment food and estimate 3-D volume for calories. Innovative, but the **density problem** makes volume→mass conversion inaccurate, so users must manually correct portions; its classifier is also trained on global data and struggles with local cuisines.

Note: Both market leaders leave a gap: **fresh, unstructured, local food measured precisely with low user friction.** That gap defines CalEyeZ.

1.2.3. Our Solution

CalEyeZ closes the gap with two coupled ideas. The **router-managed ensemble** keeps general and local knowledge in separate experts, so accuracy on standard foods is never traded away to learn new ones - directly solving catastrophic forgetting. The **BLE weight link** replaces error-prone volume mathematics with a direct, deterministic gravimetric reading. The result is a closed, automated loop: place the food, capture the image, and receive a nutrition report.

1.3. Objectives

Broad objective (מטרת-על). Develop an automated, end-to-end nutritional-analysis system that is more reliable than current market solutions by removing the human from the data-entry loop.

Specific objectives.

- **High-accuracy classification** - identify a large, diverse food vocabulary with a **validated system top-1 accuracy of at least 80%**.

- **Modular scalability** - validate a **router-based ensemble** that adds local cuisines without degrading the Global model (i.e. eliminate catastrophic forgetting).
- **Precise weight extraction** - obtain food weight through a **deterministic hardware channel** rather than visual estimation.
- **End-to-end automation** - a single raw image in, a complete nutrition report (calories, protein, fat, carbohydrates) out, via integration with the USDA database.

Note: Achieved (forward reference to Section 8). System top-1 reached **86.2%** - comfortably above the 80% target - with the Global model at **88.18%** test top-1 and a router ROC-AUC of **0.973**.

1.4. Scope and Limitations

Project scope. A **desktop / edge prototype** covering: (a) food classification via a dual-model YOLO11 ensemble and an XGBoost router; (b) weight acquisition via a BLE-enabled digital scale; (c) nutritional retrieval from the USDA API with a local fallback; and (d) a CustomTkinter GUI that fuses and displays the results.

Exclusions (non-goals).

- **Volumetric / depth-based weight estimation** - evaluated and rejected (density problem).
- **Native mobile app** - out of scope; the prototype is desktop-based.
- **Medical diagnosis or dietary prescription** - the output is informational only.
- **Multi-food plate segmentation** - one dominant item per image.
- **Real-time video** - static "snap-and-process" only.

Technical limitations.

- **Closed vocabulary** - only the trained classes are recognised; out-of-vocabulary foods are mapped to the nearest visual match.
- **Hardware dependency** - a CUDA GPU is recommended for low-latency training/inference; the shipped ONNX edge build runs CPU-only at higher latency.
- **Connectivity** - the USDA lookup needs internet (mitigated by a local JSON cache).
- **Sensing conditions** - adequate lighting for the camera and a stable BLE link for the scale.

Note: Engineering pivot recorded here for honesty: the weight subsystem was originally specified as **OCR** of the scale's 7-segment display. OCR proved unstable under glare and variable lighting, so it was **pivoted to BLE**. The OCR feasibility study is retained as documented R&D (see Section 3 and Section 9).

2. Functional Requirements

2.1. List of Requirements

The requirements below are split between the two engineering units defined in the FRS - the **Classification Engine** (the "What?") and **Weight Extraction & Data Fusion** (the "How much?") - followed by user-interface and performance requirements. Each is written so it can be objectively verified in Section 5 and Section 8.

Unit 1 - Classification Engine

1. Accept standard image formats (JPG, PNG, BMP, WEBP) and resize to the model input resolution (320 px for the Global model, 224 px for the Israeli model).
2. Run inference with two YOLO11l-cls classifiers: a Global model (132 classes) and a Local model (13 Israeli dishes + 1 open-set background class).
3. Compute, for each model and each image, the top-5 class probabilities and two uncertainty features - Shannon entropy and top-1/top-2 margin.
4. Route each image with a trained XGBoost classifier that outputs $P(\text{israeli})$ from the two models' features and selects the active expert at a threshold of 0.5.
5. Reach an aggregate system top-1 accuracy of at least 80% on the held-out test set.
6. Complete the full pipeline (preprocess $\rightarrow$ two inferences $\rightarrow$ routing) in under 2 s on a CUDA GPU (RTX 3060-class), and run CPU-only on the ONNX edge build.

Unit 2 - Weight Extraction & Data Fusion

1. Acquire the food weight in grams from the digital scale over a BLE GATT-serial link, decoding the scale's HEX notification packets.
2. Reject transient flicker/overshoot in the weight stream with a median filter and report a stable reading.
3. Query the USDA FoodData Central API with the identified food label to retrieve macronutrients per 100 g, with a local JSON database as offline fallback.
4. Compute the final nutritional values from weight and the per-100 g factors.
5. Degrade gracefully (show "service unavailable" / fall back to cache) if the API is unreachable or times out.

User Interface

1. Provide a single-window CustomTkinter dashboard with a live camera feed, a live weight reading, and an "Analyze" trigger.
2. Show the predicted class, its confidence, and the router decision (which expert won, and $P(\text{israeli})$).
3. Present the final report - calories, protein, carbohydrate, fat - for the measured weight.

Motivation. The dual-model and routing requirements exist because a single monolithic classifier suffers catastrophic forgetting when local dishes are added (see Section 3.3). The 80% target is the project's contractual accuracy floor. The BLE requirement replaces the abandoned OCR path, which failed the lighting-robustness requirement. The USDA dependency follows from the goal of reporting standardised, sourced nutritional data rather than hard-coded estimates.

2.2. Use Case Scenario

The primary scenario, "logging a meal", runs end to end as follows:

1. The user launches CalEyeZ; the scale connects over BLE and the camera feed appears.
2. The user places a single food item on the scale. The weight stream stabilises (e.g. 120 g).
3. The user clicks **Analyze**. The current frame is captured and the stable weight is latched.
4. Both classifiers run on the frame. The XGBoost router reads their features and selects the expert - for an Israeli dish it routes to the Local model, otherwise to the Global model.
5. The chosen label and the weight are fused; the USDA lookup returns macros per 100 g.
6. The dashboard shows, e.g., “Cheese Bourekas - 120 g - 350 kcal” with the macronutrient breakdown, and appends the record to the local history log.

3. Design

Note: Every design choice described in this chapter is justified against its alternatives in the next chapter, **Design Decisions and Trade-offs** (decisions D-01 to D-25), where the options, the reason for the choice, and the measured cost of each rejected alternative are set out.

3.1. System Design and Architecture

CalEyeZ is a **standalone edge-computing** system: all inference runs locally on the host PC, and only the nutritional lookup reaches out to the network. The design follows an **event-driven sensor-fusion** pattern with two parallel sensing channels - a camera for the "What?" and a BLE scale for the "How much?" - that are merged when the user triggers an analysis.

The data flow has two parallel sensing lanes - the camera lane answering **what** and the scale lane answering **how much** - that merge only at the nutrition step:

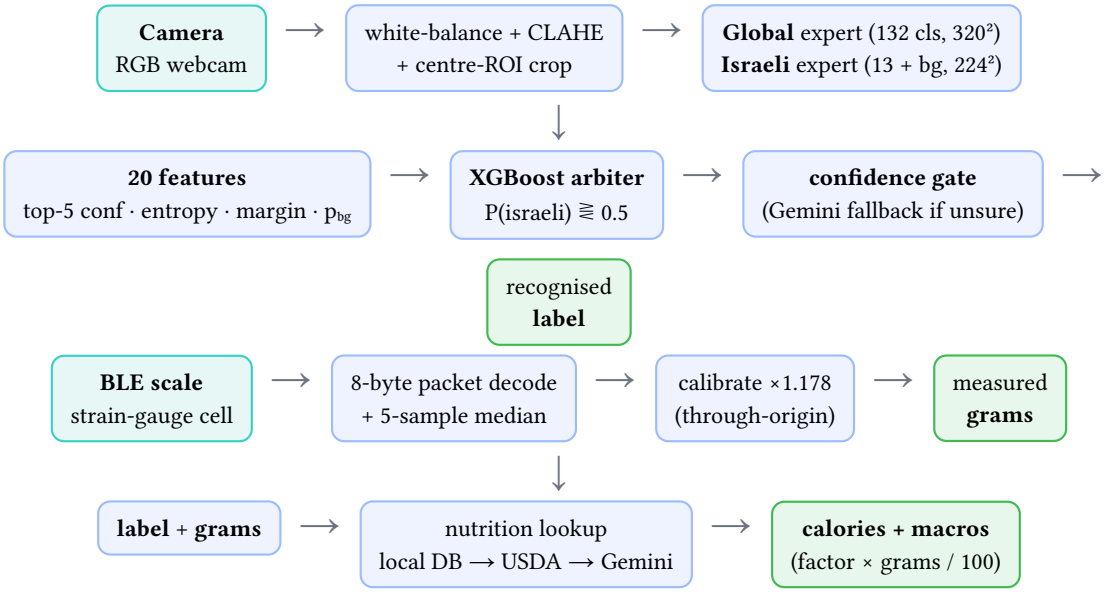


Figure 1: The CalEyeZ pipeline. The **camera lane** (top) turns a frame into a label - two CNN experts run in parallel, an XGBoost arbiter picks the right one from their confidence signals, and a gate escalates the unsure cases. The **scale lane** (middle) turns a BLE packet into calibrated grams. They meet only at the **fusion** step (bottom), where label + grams become calories. Blue = compute, teal = sensor, green = a finished quantity. Nothing infers mass from pixels - the two lanes stay separate until the arithmetic.

The central **Fusion Logic Controller** synchronises the asynchronous BLE thread with the UI event that captures the frame, runs the two models and the router, calls the nutrition resolver, and pushes the result to the GUI and the CSV log. Keeping inference on the edge gives three things the FRS asks for: privacy (images never leave the machine), low latency (no network round-trip for recognition), and offline operation for everything except the optional USDA call.

3.2. Detailed Hardware Design

The hardware is deliberately minimal because the project's contribution is algorithmic. Three elements make up the sensing and compute chain.

Digital BLE scale (load sensor). A consumer scale built around a strain-gauge load cell. The cell's resistance changes under load; an internal ADC digitises it and the scale broadcasts weight as BLE GATT notifications. We did not modify the hardware - the weight is obtained by subscribing to the notify characteristic and decoding the HEX payload (the protocol was recovered by reverse engineering, see Section 6.2). BLE was chosen over Wi-Fi or classic Bluetooth for its low power draw and because the data rate needed for a single weight value is tiny.

Camera (optical sensor). A standard 1080p USB RGB webcam supplies frames over USB 2.0/3.0. No depth, stereo or LiDAR/ToF sensor is used: the system reads weight directly, so it never needs to infer volume from the image, which is precisely the failure mode it was designed to avoid. The active/stereo depth alternatives were considered and rejected (Section 8).

Compute. Training and the low-latency demo run on a desktop with an NVIDIA RTX 3060 Ti (8 GB VRAM). The same models also run, via the ONNX export, on a CPU-only edge device at about 0.35 s per analysis.

The interfaces are all standard and off-the-shelf: USB Video Class for the camera, and the Bluetooth GATT profile for the scale. The dominant **system constraint** is the 8 GB VRAM budget, which directly shaped the training configuration - 320 px input at batch 16 fits in roughly 2.2 GB under automatic mixed precision, leaving headroom (see Section 6.2).

3.2.1. Component Selection and Justification

A BLE scale was selected over an OCR-read display because reading a 7-segment LCD under classroom glare was unreliable; BLE gives a deterministic digital value. A plain RGB webcam was sufficient because the algorithm does not rely on depth. The RTX 3060 Ti was the available GPU and its 8 GB is the binding constraint the training pipeline is tuned around.

3.3. Detailed Software Design

This is the core of the project. The software is four cooperating algorithms: two convolutional classifiers, a gradient-boosted router that arbitrates between them, and a fusion step that turns a label and a weight into nutrition. The theory behind each, and how it appears in the code, is set out below.

3.3.1. Intuition: the voice-fingerprint analogy

Before the mathematics, here is the whole system in signal-processing terms. Imagine building a system that recognises **who is speaking**. CalEyeZ is the same machine; only the input changes from a voiceprint to a food photo.

Record many people speaking the same language. A raw recording is far too much data to compare directly, so we pass each voice through a **bank of filters** - like band-pass IIR or Butterworth filters - where each filter isolates one aspect of the sound: pitch (the fundamental frequency f_0), timbre, cadence, loudness, the vowel formants. The crucial difference from classic DSP is that these filters are **not hand-designed with fixed coefficients**; they start random and are **learned from data**. In a convolutional network these learned filters are the convolution kernels, and the stack of them is the **backbone**.

After the filter bank, each voice is reduced to a short list of numbers - a **feature vector** that acts as the voice's fingerprint, or "DNA". To recognise a new voice we run it through the same filters to get its vector v, then take the **dot product** of v with one **learned template** per known speaker and add a bias, giving a raw score (a **logit**) per speaker: $z_c = w_c \cdot v + b_c$. **Softmax** turns those scores into probabilities that sum to one, and the largest wins.

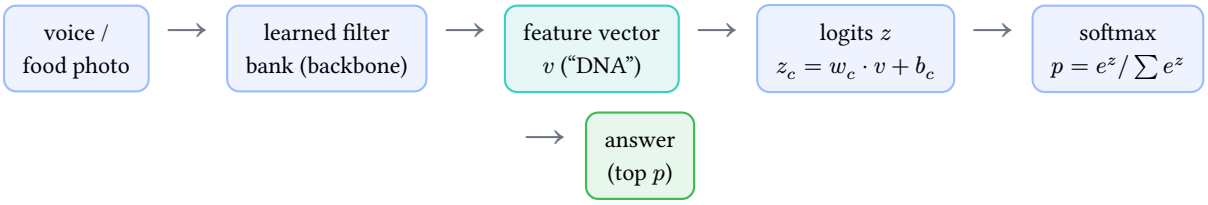


Figure 2: Recognition pipeline. The same chain identifies a speaker or a plate of food.

One subtlety keeps the analogy honest. In pure DSP you could measure pitch directly and tune a filter against that measured value. A convolutional network **cannot** - there is no ground truth for any intermediate feature. The network is graded **only** on the final answer (the class), and **backpropagation** pushes that single error backwards through every filter at each epoch, so the filters **self-organise** to extract whatever features make the final answer correct. Nobody tells them "pitch matters"; they discover it.

This also explains the ensemble. To add Israeli dishes we could retrain the whole filter bank, but the filters then drift to fit the new data and **forget** the old classes - catastrophic forgetting, which we measured and rejected (detailed below). Instead CalEyeZ keeps the original expert untouched, trains a **separate** Israeli specialist, and adds a learned **router** that decides, per image, which expert to trust. The Israeli specialist even carries a "background / not-mine" class, so when a non-Israeli food reaches it, it can signal "this isn't mine" - the strongest cue the router uses.

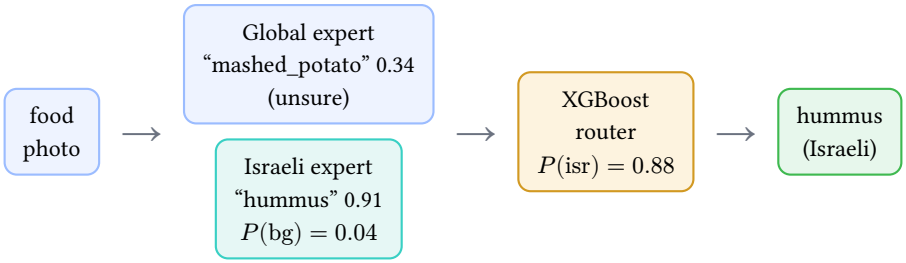


Figure 3: The router in action: both experts run, the router reads their confidence and the "not-mine" signal, then trusts the right expert.

The rest of this section makes each of these steps precise.

3.3.2. What a convolution actually computes

The "learned filter" of the analogy has an exact meaning, and for an electrical engineer it is a familiar one: a convolutional layer is a bank of **2-D FIR filters** whose coefficients are learned. A layer holding a $k \times k$ kernel K with C_{in} input channels produces each output pixel as a windowed inner product slid across the image:

$$y(i,j) = \sum_{c=1}^{C_{\text{in}}} \sum_{u=1}^{k} \sum_{v=1}^{k} K(c,u,v) \cdot x(c, i+u, j+v) + b$$

exactly the 2-D analogue of an FIR filter's $y[n] = \sum_m h[m]x[n-m]$. Three consequences follow directly from this formula and explain why CNNs work on images at all:

- **Weight sharing.** The **same** $k^2 C_{\text{in}}$ coefficients are reused at every image position, so the parameter count of a layer, $k^2 C_{\text{in}} C_{\text{out}}$, is independent of the image size. A fully connected layer on a 320^2 image would need billions of weights; the convolutional layer needs a few thousand, which is what makes the model learnable from 530 training images per class (69,487 / 132).
- **Translation equivariance.** Because the filter is slid, a feature is detected wherever it appears

  - a bourekas in the corner produces the same response as one in the centre, just displaced. The classifier does not have to re-learn each food at each position.
- **Growing receptive field.** Each layer looks only k pixels wide, but stacking layers (with stride-2 downsampling between stages) means a deep unit "sees" an exponentially larger patch of the original image. Early layers therefore learn edges and colour blobs, middle layers textures (crumb, glaze, char), and late layers whole-dish configurations - the hierarchy the voice analogy called pitch, timbre and cadence. This is directly visible in the live filter-activation visualiser on the project site, which runs the real Global backbone in the browser and shows all 11 blocks' activations for any uploaded photo.

The nonlinearity between layers (SiLU in YOLO11) is what makes the stack more than one big linear filter: without it, any depth of convolutions would collapse to a single equivalent kernel.

3.3.3. Data integrity: de-duplication and leakage control

Before any model is trained, the data has to be trustworthy, because a classifier is only as honest as the split it is measured on. The dominant threat is **data leakage**: information from the test set reaching the model at training time, which inflates the reported accuracy without any real generalisation[2]. For image classification the most common source is **near-duplicate images** - the same photo, or a lightly re-encoded/resized copy, appearing in both the training and the test split. If that happens the network can **memorise** the image during training and then "recognise" it at test time, so the score measures recall of memorised pixels, not visual generalisation.

CalEyeZ neutralises this in two passes, before the 70/20/10 re-split. The split ratio itself is a deliberate compromise: 70% training keeps enough images per class (≈ 530) to fine-tune a large backbone; the unusually large 20% validation slice exists because validation does **double duty** here - it selects the checkpoint **and** later trains the router (Section 3.3.12), so it must be big enough for both without touching test; and 10% test ($\approx$ 10k images) still gives the final accuracy a standard error of well under half a point.

Exact-duplicate removal (cryptographic hashing). Every image is reduced to a SHA-1 digest of its raw bytes,

$$h(x) = \text{SHA-1}(\text{bytes}(x)) \in \{0,1\}^{160},$$

and images sharing a digest are byte-for-byte identical. Because a cryptographic hash is collision-resistant, $h(x_1) = h(x_2)$ is a practically certain proof that $x_1 = x_2$, so exact duplicates are found in a single linear pass by bucketing on the 160-bit key rather than by an $O(N^2)$ pairwise comparison of 100k images. One canonical copy of each digest is kept; the rest are discarded. (The Israeli-model rebuild used SHA-256 for the same purpose across its raw ingest.)

Near-duplicate removal (perceptual hashing). A cryptographic hash changes completely if a single pixel changes, so it cannot catch a re-saved or slightly re-compressed copy. For that we use a **perceptual** hash - the difference hash (dHash): the image is reduced to greyscale at 9×8, and each of the 64 output bits records whether one pixel is brighter than its right- hand neighbour,

$$b_{r,c} = \mathbb{1}[I(r,c) > I(r,c+1)], \quad H_{\text{dHash}(x)} \in \{0,1\}^{64}.$$

[2] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in Data Mining: Formulation, Detection, and Avoidance," *ACM TKDD*, vol. 6, no. 4, art. 15, 2012. Near- duplicate images spanning a train/test boundary are the classic image-classification instance.

Two images are near-duplicates when their hashes are close in **Hamming distance** (the number of differing bits),

$$d_H(x_1, x_2) = \sum_{k=1}^{64} b_k^{(1)} \oplus b_k^{(2)} \leq 5,$$

with a threshold of 5 bits. Because dHash encodes **relative** brightness gradients it is invariant to global brightness/contrast shifts and mild re-compression, so it catches the copies SHA-1 misses while a tight threshold avoids collapsing genuinely distinct dishes.

The proof it worked. After de-duplication and a fresh split, the Global model's **validation and test accuracies agree to 0.02 points** (88.16% vs 88.18%). A leaking split shows the opposite signature - a test score inflated above validation - which is exactly what the **earlier** model displayed (a 4-point val/test gap). The near-zero gap is therefore not a coincidence; it is the empirical certificate that the splits are disjoint and the headline number is real.

3.3.4. The classifiers: YOLO11 classification head

Each expert is a YOLO11l-cls network[3] - a convolutional backbone followed by a classification head that outputs one logit per class. The logits $z = (z_1, \ldots, z_C)$ are turned into a probability distribution by the **softmax** function:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

so that $\sum_i p_i = 1$. Training minimises **cross-entropy** between the predicted distribution p and the one-hot ground truth y. The training script additionally requests **label smoothing** with $\varepsilon = 0.1$, which replaces the hard target with a softened one, discouraging the network from becoming over-confident:

$$y_i^{\mathrm{LS}} = (1 - \varepsilon) y_i + \frac{\varepsilon}{C}, \quad \mathcal{L} = -\sum_{i=1}^{C} y_i^{\mathrm{LS}} \log p_i$$

Note: A documentation-versus-runtime finding, recorded honestly. While auditing this book against the actual run artifacts, we found that the Ultralytics classification trainer (v8.3) computes plain `F.cross_entropy` and **silently ignores** the `label_smoothing` argument - the run's own `args.yaml` records it as unset. The parameter is therefore in the script as **intent**, but the smoothing term was **not active** in the shipped models. The model's measured calibration (which the arbiter consumes) rests on dropout, the heavy augmentation, and small-batch gradient noise - and this is knowable precisely because every claim in this book was re-checked against the committed artifacts rather than the source code alone.

The learning rate follows a **cosine schedule**[4] from lr_0 (the initial learning rate) down to $\mathrm{lr}_f \cdot \mathrm{lr}_0$ over T epochs, which lets the optimiser settle into a sharper minimum than a flat rate:

[3] The YOLO family originates with J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," *CVPR*, 2016, arXiv:1506.02640; CalEyeZ uses the classification variant of the current release, G. Jocher and J. Qiu, *Ultralytics YOLO11*, 2024, github.com/ultralytics/ultralytics. The backbone is initialised from ImageNet features (J. Deng et al., "ImageNet: A Large-Scale Hierarchical Image Database," *CVPR*, 2009).

[4] I. Loshchilov and F. Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts," *ICLR*, 2017, arXiv:1608.03983. The cosine decay lets the effective step shrink smoothly to near zero, which anneals the optimiser into a flatter, better-generalising basin than a fixed rate.

$$\mathrm{lr}(t) = \mathrm{lr}_f \cdot \mathrm{lr}_0 + \frac{1}{2}(\mathrm{lr}_0 - \mathrm{lr}_f \cdot \mathrm{lr}_0)\left(1 + \cos\left(\pi \frac{t}{T}\right)\right)$$

The schedule is preceded by a 5-epoch **warmup** in which the learning rate ramps up from near zero. The reason is specific to transfer learning: at epoch 0 the classification head is freshly initialised (random), so its gradients are large and essentially noise; taking full-size steps then would scramble the pretrained ImageNet backbone before the head has learned anything. Warming up lets the head settle first, protecting the very features transfer learning is supposed to reuse.

The Global model is trained on the cleaned 132-class dataset at 320 px; the Israeli model on 13 dishes plus an open-set **background** class at 224 px. The full training call (Global model) is reproduced here - note the cosine schedule, dropout and the deliberately heavy geometric/photometric augmentation that buys classroom robustness:

```
model = YOLO("yolo11l-cls.pt")
results = model.train(
    data=DATASET_DIR, task="classify",
    epochs=150, imgsz=320, batch=16, device=0,
    amp=True,
    optimizer="AdamW", lr0=0.001, lrf=0.01, cos_lr=True,   # cosine LR decay
    momentum=0.937, weight_decay=0.0005, warmup_epochs=5,
    dropout=0.2, label_smoothing=0.1,                      # anti-overconfidence
    patience=30,
    hsv_h=0.02, hsv_s=0.75, hsv_v=0.50,                    # photometric aug
    degrees=20.0, translate=0.15, scale=0.60, shear=8.0,   # geometric aug
    flipud=0.15, fliplr=0.50, erasing=0.40, mixup=0.10,
)
```

3.3.5. How learning works: gradient descent and backpropagation

Training means searching for the network weights θ (every convolution kernel and bias) that minimise the loss $\mathcal{L}(\theta)$. The search is **gradient descent**: repeatedly nudge the weights a small step in the direction that most reduces the loss, namely the negative gradient,

$$\theta \leftarrow \theta - \eta \cdot \nabla_\theta \mathcal{L}(\theta)$$

where η is the learning rate (Section 3.3.4). The gradient $\nabla_\theta \mathcal{L}$ says how the loss changes with each individual weight. Computing it for a deep network naively would be hopeless, so it is computed by **backpropagation**: the chain rule applied layer by layer, from the output back to the input. If layer l produces $z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$ followed by an activation $a^{(l)} = f\left(z^{(l)}\right)$, the error signal $\delta^{(l)} = \partial \mathcal{L} / \partial z^{(l)}$ is obtained from the next layer's error by

$$\delta^{(l)} = \left(W^{(l+1)}\right)^\top \delta^{(l+1)} \circ f'\left(z^{(l)}\right), \qquad \partial \frac{\mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} \left(a^{(l-1)}\right)^\top$$

with $\circ$ the element-wise product. In words: each layer receives the blame for the final error from the layer above it, scales it by how sensitive its own activation was, and passes a share back to the layer below. This is exactly why the analogy in Section 3.3.1 holds: the "filters" are never told a correct intermediate value: they are corrected only by the share of the final error that flows back to them.

CalEyeZ uses the **AdamW** optimiser[5], an adaptive variant that keeps running estimates of the first and second moments of the gradient (m_t, v_t) and takes a per-weight step

$$\theta \leftarrow \theta - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} - \eta\lambda\theta$$

so directions with consistently small gradients still make progress, and the decoupled $-\eta\lambda\theta$ term is weight decay (regularisation). One such update is one **step**.

3.3.6. Batches, epochs, and the noise in the gradient

We cannot use the whole training set for every step (it does not fit in memory, and it would be wasteful), so each step uses a random **mini-batch** of B images and estimates the gradient as an average over them:

$$\hat{g} = \frac{1}{B}\sum_{i=1}^{B} \nabla_\theta \mathcal{L}_i \approx \nabla_\theta \mathcal{L}$$

This estimate is unbiased, and its variance falls as the batch grows:

$$\mathrm{Var}(\hat{g}) \propto \frac{\sigma^2}{B}$$

That single relation drives the whole batch-size trade-off. A **large** batch gives a smooth, low-noise gradient and uses the hardware efficiently, but each step costs more memory and there are fewer steps per epoch. A **small** batch gives a noisy gradient, but the noise itself is a mild regulariser (it helps escape sharp minima), and the small batch forced by our 8 GB VRAM budget is therefore not purely a cost. You get many more updates per pass. One **epoch** is one full pass over the training set, so it contains N/B steps. For our Global model, $N = 69\{,\}491$ training images at $B = 16$ give about $4\{,\}343$ steps per epoch, and training ran for 116 epochs before early stopping. As Section 3.3.8 explains, our batch size was not a free choice: it was capped by GPU memory, so we accepted a noisier gradient in exchange for a higher image resolution.

3.3.7. Training loss, validation loss, and why we need a validation set

The loss measured on the images the network is **training** on (the **training loss**) is a biased view of quality, because the network can simply memorise those images. To see whether it has learned something that **generalises**, we measure the loss on a separate **validation set** that is never used to update the weights:

$$\mathcal{L}_{\text{train}} = \frac{1}{N_{\text{tr}}}\sum_{i \in \text{ train}} \mathcal{L}_i, \qquad \mathcal{L}_{\text{val}} = \frac{1}{N_{\text{val}}}\sum_{i \in \text{ val}} \mathcal{L}_i$$

[5]The decoupled-weight-decay variant of Adam: I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," *ICLR*, 2019, arXiv:1711.05101, which separates the $-\eta\lambda\theta$ regularisation term from the adaptive gradient step.

The difference $\mathcal{L}_{\text{val}} - \mathcal{L}_{\text{train}}$ is the **generalisation gap**. While both fall, the model is genuinely learning; once the training loss keeps falling but the validation loss starts **rising**, the model is **overfitting**: memorising training quirks that do not transfer. This is precisely the behaviour our run shows (see Figure 3). The validation loss bottomed out around epoch 50 (validation loss 0.558 against a training loss of 0.346, a gap of 0.21), after which the training loss kept dropping to 0.087 while the validation loss drifted back up to 0.667, widening the gap to 0.58. Because we keep the weights that were best on validation rather than the last ones, the **deployed** `best.pt` is frozen near that peak (epoch 73, validation top-1 88.16%) and is unaffected by the later overfitting. **Early stopping** with patience 30 then halts training once the validation metric has not improved for 30 epochs, which is why 150 planned epochs ended at

116. This is also why our validation and test scores agreeing (88.16% vs 88.18%) is meaningful:

the validation set honestly predicted unseen performance.

3.3.8. The hardware budget: how VRAM and CPU shaped the results

The choices above were constrained by hardware, and the constraints are quantifiable.

GPU memory (training). During training the GPU must hold the model weights and, far larger, the **activations** of every layer for the whole batch, because backpropagation needs them. Activation memory scales roughly as

$$M_{\text{act}} \propto B \times H \times W \times C$$

that is, linearly with batch size and with the image area $H \times W$. Raising the input from 224^2 to 320^2 multiplies the area, and hence the activation memory, by $(320/224)^2 \approx 2.04$. To stay inside the 8 GB budget of the RTX 3060 Ti we therefore halved the batch from 32 to 16, which divides activation memory by two and roughly cancels the increase (measured footprint about 2.2 GB under mixed precision). The engineering point is that resolution and batch size trade against each other under a fixed memory ceiling $B \times H \times W \times C \leq M$, and for fine-grained food the **resolution** is the stronger accuracy lever, so we spent the budget there and accepted the noisier small-batch gradient. The same ceiling is why we used the YOLO11l backbone rather than the larger YOLO11x: a bigger model plus 320 px plus a usable batch would not co-exist in 8 GB. In short, the hardware did not corrupt the results, but it bounded them: a larger memory budget would permit a larger backbone or batch and is one of the few remaining levers (Section 8) now that the system is model-limited.

CPU (edge inference). On the CPU-only edge build there is no such training cost, but inference latency is set by throughput: roughly

$$t_{\text{infer}} \approx \frac{\text{FLOPs}}{\text{throughput}}$$

A CPU delivers on the order of tens of GFLOP/s of usable throughput against a GPU's several TFLOP/s, hundreds of times less, which is why the same model takes about 0.35 s per analysis on CPU (after the ONNX optimisation) versus milliseconds on the GPU. That latency is fine for the "snap and analyse" workflow but rules out real-time video, and it is the reason the edge path was exported to ONNX (1 to 3 s under the plain PyTorch CPU path was too slow). It also forced one functional compromise: the 512-dimensional embedding used for image tagging is not available through the ONNX classification head, so on the edge build it is stored as zeros. As with VRAM, the limit did not poison correctness (top-1 parity with the GPU model is exact), but it shaped what the edge product can and cannot do.

3.3.9. Why an ensemble: catastrophic forgetting

The natural design - one network for every food - fails in practice. When a single model that already knows international foods is fine-tuned to add Israeli dishes, gradient updates that lower the loss on the new classes overwrite the weights that encoded the old ones. The network's accuracy on the original classes collapses even as it learns the new ones; this is **catastrophic forgetting**. Because the two label spaces are disjoint, CalEyeZ avoids the problem entirely: it keeps two independent experts and learns **which one to trust per image**. This is a *mixture-of-experts* arrangement in the classical ensemble-learning sense - a gating function (our arbiter, learned separately from the experts) selects among specialists whose errors are decorrelated by construction (disjoint label spaces). Adding a new cuisine then means training a new expert, never disturbing the Global model.

3.3.10. The uncertainty features

The router does not see the image - it sees how each model **reacted** to the image. Two information-theoretic features summarise that reaction. The first is **Shannon entropy** of the full probability vector, which measures how spread-out (uncertain) a prediction is:

$$H(p) = -\sum_{i=1}^{C} p_i \log p_i$$

A confident prediction concentrates mass on one class and has low entropy; a confused one spreads mass and has high entropy. The second is the **top-1/top-2 margin**, the gap between the best and second-best class:

$$\text{margin} = p_{(1)} - p_{(2)}$$

where $p_{(1)} \geq p_{(2)}$ are the two largest probabilities. A large margin means the model is decisive; a small one means it is torn between two classes. Both are computed directly from the model's probability output:

```
full = probs.data.cpu().numpy().astype(np.float64)
full = np.clip(full, 1e-12, 1.0)               # avoid log(0)
entropy = float(-(full * np.log(full)).sum())  # Shannon entropy H(p)
margin  = float(top5_conf[0] - top5_conf[1])   # top-1 minus top-2
p_bg    = float(full[bg_idx]) if bg_idx is not None else 0.0   # P(background)
```

The single most informative feature turned out to be `i_p_background` - the Israeli model's probability that the image is **not** one of its dishes. Because the background class was trained on copies of non-Israeli food, this probability sits near 0.5 on global food and near 0.03 on genuine Israeli food, giving the router a clean "this is not mine" signal. It is the top feature by gain in the trained router and is the main reason routing improved from ROC-AUC 0.93 to 0.97 (see Section 8) - the gain came from **adding information**, not from tuning the decision threshold.

3.3.11. The 14th class as an out-of-distribution detector

The background class deserves a formal treatment, because it is the single most important algorithmic idea in the ensemble and it rests on a well-studied failure mode of deep networks.

The problem: softmax is not a probability of being in-distribution. A closed-set classifier is trained only on its C known classes, so its softmax is a distribution **conditioned on the input being one of those classes**. Fed an out-of-distribution (OOD) input - here, a non-Israeli food shown to the 13-class Israeli model - the network has no "none of the above" option and must place its unit mass of probability on the known classes anyway. Deep ReLU networks are moreover known to produce **arbitrarily high** softmax confidence on inputs far from the training data[6]. This is exactly what we measured: the closed-set Israeli model fired $p_{(1)} \geq 0.5$ on almost **any** food (it "shouted hummus" on beige global dishes), so its raw confidence carried almost no information about whether the image was actually Israeli.

Why the naive detector fails. The standard baseline for OOD detection is the maximum softmax probability (MSP): flag an input as OOD when $\max_i p_i$ is low[7]. Because our closed-set model is overconfident on OOD food, its MSP is high on precisely the inputs it should reject, so MSP is useless here. More elaborate post-hoc scores exist - the Mahalanobis distance in feature space and free-energy (energy-based) scores - and are noted as future work, but they add a second inference-time computation and a threshold to tune on held-out OOD data.

Our mechanism: an explicit background class (Outlier Exposure). Instead of a post-hoc score we change the **training objective** itself, adding an explicit $(C+1)$-th "background / not-mine" class and populating it with real global-domain look-alikes. This is the *Outlier Exposure* principle[8]: exposing the network to a curated outlier set during training teaches it to route OOD probability mass to a dedicated sink rather than onto a real class. The softmax now spans $C+1$ classes,

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{C+1} e^{z_j}}, \quad i \in \{1, \ldots, C, \mathrm{bg}\},$$

and the quantity p_{bg} becomes a **learned, calibrated** estimate of $P(\text{input is not Israeli})$. Empirically it separates almost linearly: $p_{\mathrm{bg}} \approx 0.50$ on global food versus ≈ 0.03 on genuine Israeli food.

[6] M. Hein, M. Andriushchenko, and J. Bitterwolf, "Why ReLU Networks Yield High-Confidence Predictions Far Away from the Training Data," *CVPR*, 2019, arXiv:1812.05720, prove this overconfidence is intrinsic to piecewise-linear networks.

[7] D. Hendrycks and K. Gimpel, "A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks," *ICLR*, 2017, arXiv:1610.02136.

[8] D. Hendrycks, M. Mazeika, and T. G. Dietterich, "Deep Anomaly Detection with Outlier Exposure," *ICLR*, 2019, arXiv:1812.04606.

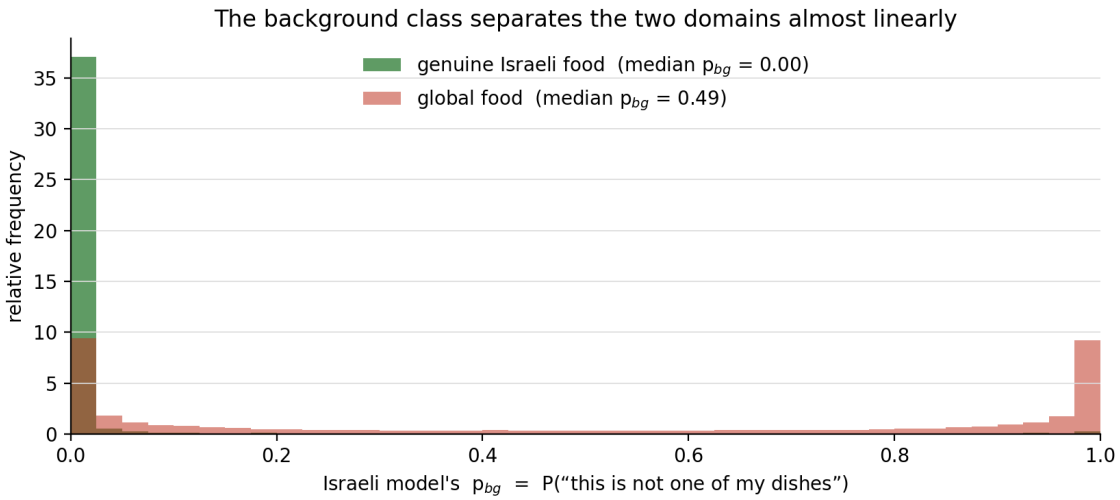


Figure 4: The background class in action, over all 32,136 held-out rows (`datasets/arbiter_dataset.csv`). Genuine Israeli food (green) piles up at $p_{\text{bg}} \approx 0$ - the specialist confidently claims it - while global food (red) sits high, with a heavy mass near 1 (median 0.49). One number, computed for free from the Israeli model's own softmax, tells the arbiter "this is / isn't mine," which is why it is the top routing feature by gain and lifted the routing ROC-AUC from 0.933 to 0.973.

How the arbiter exploits it. The feature p_{bg} (`i_p_background`) is handed to the XGBoost arbiter, where it becomes the top feature by gain (0.289). It lets the arbiter **dynamically re-weight** the two experts per image rather than trusting a naive average or a single monolithic model: a high p_{bg} is the Israeli expert declaring "this is not mine," which the arbiter reads as strong evidence to route to the Global model, while a low p_{bg} paired with a confident, low-entropy Israeli prediction is strong evidence to trust the specialist. In information terms the background class converts the previously uninformative Israeli-confidence channel into a high-signal one, which is why the routing ROC-AUC rose from 0.933 to 0.973 purely from **added information** rather than from moving the decision threshold (a change we separately proved to be zero-sum; Section 8).

3.3.12. The router: gradient-boosted decision trees (XGBoost)

The router is a binary classifier whose target is the image **domain**: $y = 1$ if the image is Israeli, 0 if global. It never sees the domain label at inference - it predicts it from 20 features (each model's top-5 confidences, entropy, margin, the background probability, and a handful of interaction terms such as the confidence gap and ratio between the two models).

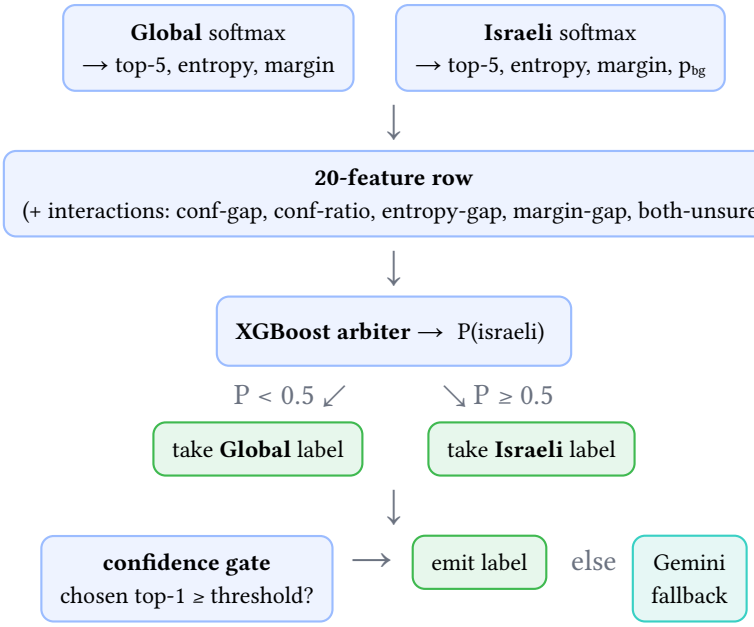


Figure 5: What the arbiter actually decides at inference. The two experts' softmax vectors become a 20-number row (no pixels); XGBoost turns it into P(israeli); the 0.5 split picks which expert's label to keep; and a confidence gate emits it only if the chosen top-1 clears its threshold, otherwise escalating to the Gemini fallback. This is the decision logic behind the single "arbiter" box in the pipeline figure.

XGBoost[9] builds an **additive ensemble of regression trees**. The model's raw output for an image x is the sum of K trees plus a bias, and the probability is the logistic (sigmoid) of that score:

$$\text{logit}(x) = \sum_{k=1}^{K} f_k(x), \qquad P(\text{israeli} \mid x) = \frac{1}{1 + e^{-\text{logit}(x)}}$$

Each new tree is fit to reduce a regularised objective that balances fit against complexity, which is what keeps a boosted ensemble from overfitting:

$$\mathcal{L} = \sum_{n} \ell(y_n, \hat{y}_n) + \sum_{k=1}^{K} \Omega(f_k), \qquad \Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

where T is the number of leaves in a tree and w_j the leaf weights. The classes are imbalanced (far more global than Israeli images), so the positive class is up-weighted by

$$\text{scale\_pos\_weight} = \left(\frac{n_{\text{neg}}}{n_{\text{pos}}} \right)^{\alpha}, \quad \alpha = 0.5$$

The $\alpha = 0.5$ exponent is a deliberate compromise: $\alpha = 1$ fully balances the classes but floods the router with false "Israeli" routes; $\alpha = 0.5$ recovers Israeli recall without wrecking global precision. The training call:

[9] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," *KDD*, 2016, arXiv:1603.02754. Boosted trees remain the strongest general method on tabular feature vectors such as ours.

```
n_pos, n_neg = int(ytr.sum()), int((ytr == 0).sum())
spw = (n_neg / n_pos) ** 0.5                  # scale_pos_weight, alpha = 0.5

clf = xgb.XGBClassifier(
    n_estimators=400, max_depth=5, learning_rate=0.05,
    subsample=0.8, colsample_bytree=0.8,
    scale_pos_weight=spw, eval_metric="auc",
    min_child_weight=3, reg_lambda=1.0,       # lambda in the objective above
)
clf.fit(Xtr, ytr)                             # train on VAL-split rows
proba = clf.predict_proba(Xte)[:, 1]          # evaluate on TEST-split rows
route_israeli = proba >= 0.5
```

The hyperparameters follow the standard bias-variance logic for boosting on a small feature set. `max_depth=5` caps each tree at interactions of at most five features - deep enough to express the confidence-gap interactions that matter, shallow enough not to memorise individual rows. A **low** learning rate (0.05) with **many** trees (400) is the classic shrinkage recipe: each tree corrects only 5% of the remaining error, so no single tree can overfit a quirk, and the ensemble averages over hundreds of weak corrections. `subsample=0.8` and `colsample_bytree=0.8` train each tree on a random 80% of rows and features (stochastic boosting), decorrelating the trees; `min_child_weight=3` refuses leaves supported by fewer than 3 rows; and `reg_lambda=1.0` is the λ term of the objective above, shrinking leaf weights toward zero. None of these were exotically tuned - they are conservative defaults, and the router's headroom analysis (Section 8) shows the system is limited by the experts, not by router capacity.

Our evaluation is leakage-free, and we verify it. The Global model's validation and test accuracy agree (88.16% vs 88.18%), which is the signature of clean, non-leaking splits. The router **is trained only on rows from the validation split and scored only on rows from the held-out test split** - never on the Global model's own training images, where confidence is artificially near 100%. Using only val/test features gives the router realistic inputs, and the test rows are never seen during router training, so the reported system accuracy is honest.

The router is also **explainable per prediction**. XGBoost can decompose a single decision into per-feature contributions (SHAP[10]), which sum exactly to the logit. The demo uses this to show **why** a given image was routed:

```
booster = arbiter.get_booster()
contribs = booster.predict(xgb.DMatrix(X, feature_names=FEATS),
                           pred_contribs=True)[0]   # n_features + bias
# each contrib is log-odds pushed toward 'Israeli' (>0) or 'Global' (<0);
# they sum to logit, and P(israeli) = 1 / (1 + e^-logit)
```

3.3.13. Fusion: from label and weight to nutrition

Once the label and weight are known, nutrition is deterministic. USDA returns macronutrient factors **per 100 g**; the report scales them linearly by the measured mass:

[10] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," *NeurIPS*, 2017, arXiv:1705.07874. SHAP values are the unique feature attributions satisfying local accuracy, missingness and consistency, and they sum exactly to the model's logit; XGBoost computes them exactly in polynomial time via TreeSHAP.

$$\text{value}_{\text{total}} = \text{factor}_{\text{per 100 g}} \times \frac{\text{weight}_g}{100}$$

The resolver tries the local database first, then USDA, then a fallback estimate, so the report is never blank:

```
def nutrition(label, grams):
    ratio = max(grams, 0) / 100.0            # weight_g / 100
    desc, cal, pro, carb, fat = resolve(label)   # local DB -> USDA -> estimate
    return {"cal": int(cal * ratio), "pro": round(pro * ratio, 1),
            "carb": round(carb * ratio, 1), "fat": round(fat * ratio, 1)}
```

The weight itself is median-filtered over the last few BLE decodes to reject flicker and overshoot before it is latched at the moment of analysis.

3.4. User Interface Design

The GUI is a single CustomTkinter window so a first-time user can learn it in minutes. It has a live camera panel (to frame the food), a live weight readout and a small weight-vs-time graph (to confirm the reading is stable before analysing), one prominent **Analyze** button, and BLE/camera status indicators. After an analysis it shows the recognised label with its confidence, the router decision ($P(\text{israeli})$ and which expert won), and four result cards showing calories, protein, carbohydrate and fat for the measured weight. A camera-switch control handles machines with both an internal and a USB camera.

3.5. List of CAD / Software Tools

The project is software-led, so the "CAD" tools are the modelling and development environment: **Python 3.10** as the language; **Ultralytics YOLO11** (PyTorch) for training and inference of the two classifiers; **Albumentations** for the custom blur augmentation; **XGBoost** and **scikit-learn** for the router and its metrics; **ONNX Runtime** for the torch-free edge build; **OpenCV** and **Pillow** for image handling; **Bleak** for BLE communication; and **CustomTkinter** for the GUI.

4. Design Decisions and Trade-offs

Every engineering choice in CalEyeZ was made against alternatives, and wherever possible the alternative was **measured** rather than assumed. This chapter is a decision register: each entry states the options, the choice, what the rejected options would have cost (with evidence from our own runs where we have it), and the consequence. The recurring theme is that many of our "alternatives" are not hypothetical - we ran them and kept the numbers.

4.1. Problem Framing and Model Architecture

4.1.1. D-01 · Task framing: whole-image classification, not detection or segmentation

Options. Object detection (YOLO-detect, one box per food), semantic segmentation (per-pixel masks), or single-label image classification.
Chosen. Classification of a central region of interest.
Cost of alternatives. Detection and segmentation both require **bounding-box or mask labels**, which our data (single-dish photos) does not have and which are expensive to annotate for 145 classes; they also solve a harder problem (localisation) we do not need, because the user frames one food and the scale already isolates the item. We even tested implicit segmentation at demo time (GrabCut, border-colour cropping) and it made recognition **worse** - a clean 96% hummus fell to 92% malawach - because the classifier is already background-robust (99.97% on a pepper against wood/dark backgrounds).
Consequence. A simpler labelling task, a smaller model, and a center-ROI pipeline that matches how the product is actually used.

4.1.2. D-02 · Framework: Ultralytics YOLO11 over hand-rolled PyTorch/timm or TensorFlow

Options. Raw PyTorch + timm, TensorFlow/Keras, or Ultralytics.
Chosen. Ultralytics YOLO11.
Cost of alternatives. A hand-rolled training loop re-implements data loading, augmentation, EMA, cosine scheduling, checkpointing and export that Ultralytics already provides and that are easy to get subtly wrong; TensorFlow would not share tooling with the ONNX edge path we needed.
Consequence. One reproducible script per stage and a first-class `export` to ONNX, which the whole edge and browser story depends on.

4.1.3. D-03 · Backbone: YOLO11l-cls, ImageNet-pretrained, fully fine-tuned

Options. (a) train from scratch; (b) a smaller `n/s/m` or larger `x`; (c) a different backbone family (ResNet50, EfficientNet, ConvNeXt-T, ViT); (d) freeze the backbone and train only the head.
Chosen. The `l` (large) backbone, initialised from ImageNet, with **all** layers trainable (full fine-tuning).
Why full fine-tuning over a frozen backbone. This is the central training decision, so it is justified explicitly. A frozen backbone reuses ImageNet features unchanged and trains only the classifier head; full fine-tuning also lets the convolutional layers re-adapt their features to food. The two differ in a **bias-variance** sense: freezing is faster and regularises, but caps the achievable accuracy; full fine-tuning reaches a higher ceiling at the cost of training time and a larger dataset to avoid overfitting. For **food recognition specifically**, full fine-tuning is the established choice: food classes are separated by fine **texture** - crumb, glaze, grain, char - so the convolutional layers must re-adapt to the food domain rather than stay frozen on generic ImageNet edges and shapes. We adopted this known result rather than spend scarce lab compute re-deriving it in a controlled A/B, which would cost days of GPU time to reproduce a settled finding.
Cost of the other alternatives.

- **From scratch:** discards the ImageNet prior and needs far more than our 530 training images/class.
- **x (larger):** exceeds the **8 GB VRAM** budget at 320 px, batch 16.
- **ConvNeXt-T / ViT:** a plausibly higher ceiling (we keep this in Future Work) but heavier to train and to run on a phone CPU at the edge.

Consequence. 88.18% top-1 on the held-out test set, consistent with the literature's finding that full fine-tuning maximises food-recognition accuracy.

4.1.4. D-04 · Input resolution: 320 px (Global) and 224 px (Israeli)

Options. A single common size, or larger inputs for accuracy.
Chosen. 320 px for the Global model, 224 px for the Israeli model.
Cost of alternatives. Convolution cost grows with the **square** of the input edge (Section 7.6), so a larger input multiplies both training time and edge latency; a smaller input loses the fine texture that separates look-alike dishes. 320/224 were the largest sizes that fit the 8 GB VRAM budget at a usable batch.
Why the two models get different sizes. The asymmetry follows the difficulty of each model's task. The **Global** model must separate **132** classes riddled with fine-grained look-alike pairs (mousse vs cake, gnocchi vs ravioli) whose only distinguishing evidence is **texture** - crumb, glaze, grain - and texture is exactly what extra resolution buys, so the Global model gets the largest input the VRAM budget allows ($(320/224)^2 \approx 2 \times$ the pixels). The **Israeli** specialist solves a far easier 14-way problem over visually distinct dishes, so 224 px - the native resolution of the ImageNet pretraining, where the transferred backbone features fit best - is already sufficient (88.9% top-1 for the shipped 13 + background model), and raising it would spend compute without a discrimination problem to solve. The saving is not free-floating either: the arbiter design runs **both** experts on **every** image (Decision D-10), so their inference costs **add** - keeping the second model at 224 px is what keeps the double-inference latency budget affordable on the CPU edge build. The smaller input also halves the Israeli model's activation memory, which is what allowed its larger batch (32 vs 16) on the same GPU.
Consequence. Resolution spent where the discrimination problem actually is; affordable training; sub-second double-inference on the edge; and no train/inference resolution mismatch (the demo and arbiter feed each model exactly its training size).

4.2. Dataset and Training Regime

4.2.1. D-05 · Curate 158 classes down to 132, with leakage-free re-splitting

Options. Train on the raw 158-folder collection as-is, or clean it first.
Chosen. Merge visually identical sub-classes, delete disallowed/noisy ones, move 14 dishes to the Israeli expert, then de-duplicate (exact SHA-1 + perceptual dHash) and re-split 70/20/10.
Cost of alternatives. The raw set had cross-split duplicates that **inflated** the old model's test score; training on it would have reported an accuracy we could not defend.
Consequence. Validation and test agree (88.16% vs 88.18%), which is the proof the splits are clean and the number is real.

4.2.2. D-06 · Augmentation: custom blur, label smoothing, dropout; mosaic off

Options. Default detection augmentation, or a classification-appropriate set.

Chosen. Heavy colour/geometry augmentation plus a custom Albumentations blur (to mimic classroom motion blur), dropout 0.2, and label smoothing $\varepsilon = 0.1$ requested in the script (found during the book audit to be silently ignored by the Ultralytics classify trainer - see the note in Section 3.3.4); mosaic **disabled**.

Cost of alternatives. Mosaic augmentation (four images stitched) is designed for detection and corrupts a single-label classification target; leaving it on would teach the wrong objective.

The lighting trade-off, made on purpose. The heaviest augmentation is photometric, because harsh and variable lighting is the binding real-world constraint. On **every** batch each image is re-lit at random: brightness $V \leftarrow V \cdot U(0.5, 1.5)$ (via hsv_v $= 0.50$, i.e. up to 50% darker or brighter - the "dark plate" case), colour temperature $H \leftarrow (H \cdot U(0.98, 1.02)) \bmod 180$ (hsv_h $= 0.02$), and saturation $S \leftarrow S \cdot U(0.25, 1.75)$ (hsv_s $= 0.75$); on top of that the custom blur ($p = 0.35$) and random erasing (0.40). Over 116 epochs each photo is seen 116 times, each under a different random lighting, so the network learns **lighting-invariant** features by construction. This is a deliberate **robustness-over-peak-accuracy** trade-off: a moderate-augmentation retrain scores about **90%** on the clean test split but is measurably more brittle in the field, whereas the heavy-augmentation model scores **88.18%** and holds up under real conditions. We accepted the 2-point clean-test cost because the deployment target is a kitchen, not a lab.

Consequence. A model robust to lighting, blur and pose, with calibrated confidences for the arbiter. The choice is validated by our own numbers: the near-zero validation-test gap (88.16% vs 88.18%) shows no overfitting - exactly what heavy augmentation buys - and the field validation (30/40 = 75%, 85% expanded) confirms it transfers to unseen real photos.

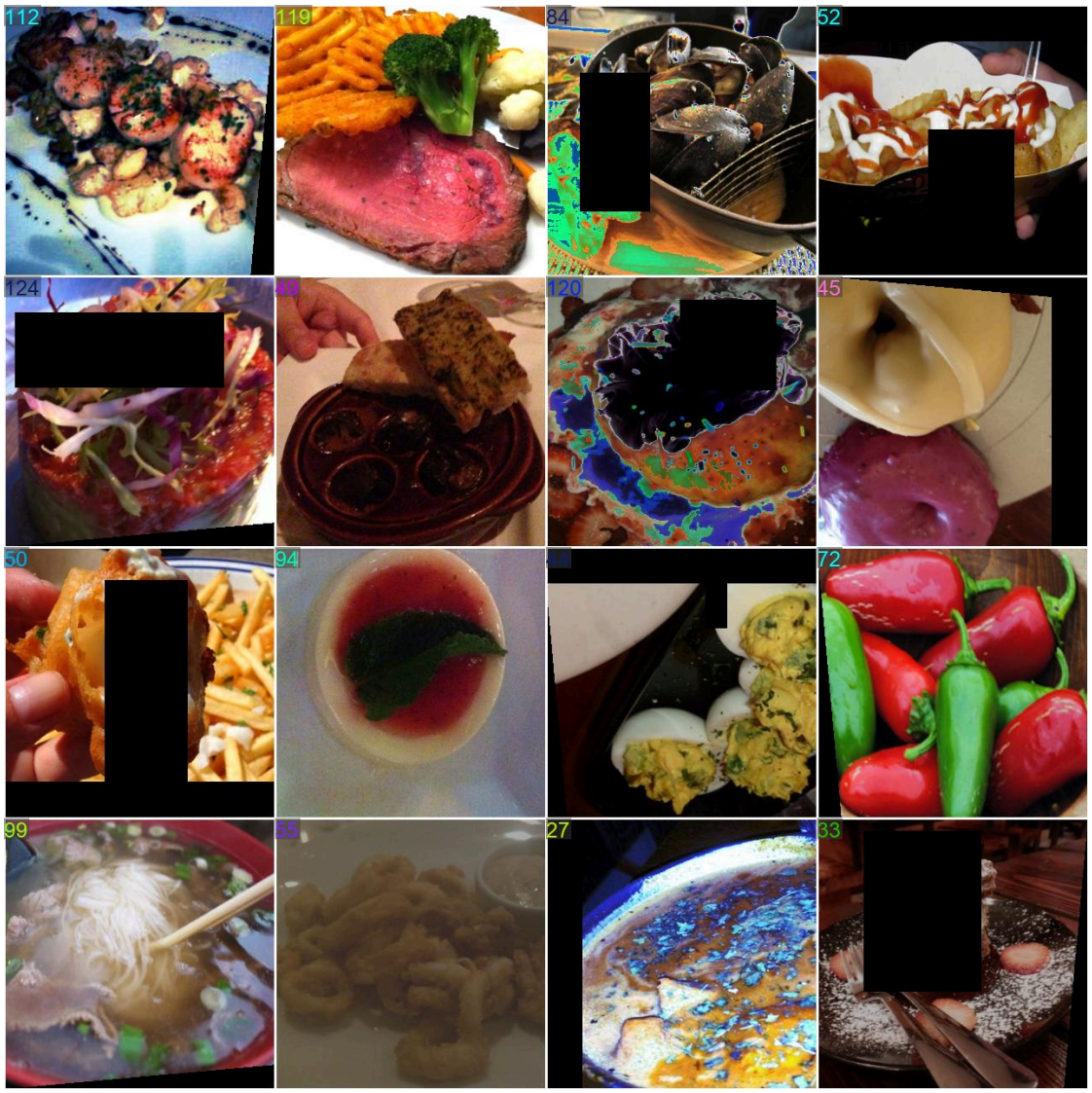


Figure 6: One real training batch fed to the Global model, exactly as the network sees it. The augmentation described above is visible on the actual data: images are re-lit at random (note the dark and the over-bright plates), colour-shifted, rotated and letter-boxed onto black, and each carries a **random-erasing** cut-out (the black rectangles) that forces the model off any single discriminative patch. The coloured integers are class indices. This is the mechanism behind the lighting-invariance claim - every epoch re-draws these transforms, so no photo is ever seen twice the same way.

4.2.3. D-07 · Batch size 16 (Global) / 32 (Israeli): a VRAM decision

Options. Larger batches for a smoother gradient.

Chosen. 16 at 320 px, 32 at 224 px.

Cost of alternatives. Batch size is bounded by the **8 GB VRAM** of the RTX 3060 Ti; a larger batch at 320 px overflows. The gradient noise from a small batch is partly a **feature** (mild regularisation), and the cosine schedule absorbs the rest.

Consequence. Training fits the hardware and still converges cleanly.

4.2.4. D-08 · Open-set "background" class in the Israeli model (V2)

Options. Leave the 13-class Israeli model as a closed set, or add a 14th "not-Israeli" class.
Chosen. Add a background class built from global-domain look-alikes.
Cost of alternatives. The closed-set model was **overconfident on everything** - it fired ≥ 0.5 on almost any food, so its confidence was not a usable routing signal (it "shouted hummus" on beige dishes).
Consequence. $P(\text{background})$ separates cleanly (mean 0.50 on global food vs 0.03 on Israeli) and became the arbiter's single most informative feature (gain 0.289), lifting routing AUC 0.933 to 0.973 and the whole system from 83.75% to 86.16%.

4.3. The Ensemble and the Arbiter

4.3.1. D-09 · Two specialist models instead of one unified classifier

Options. One model over all 145 classes, or two domain experts (Global 132 + Israeli 13) plus a selector.
Chosen. Two experts.
Cost of alternatives. A single model trained on the combined set suffered **catastrophic forgetting** of the rare Israeli dishes, which are swamped by the large global set; the Israeli foods are also culturally specific and benefit from a dedicated, balanced expert. The class sets are **disjoint** (Global $\cap$ Israeli $= \varnothing$), so on any one image only one expert can be right - which is precisely what makes a selector meaningful.
Consequence. The Israeli-domain accuracy ceiling rose to 92-93%, unreachable by the diluted single model.

4.3.2. D-10 · An arbiter that adjudicates after both experts run, not an up-front router

Options. (a) a router that picks one expert from the image **before** inference; (b) an arbiter that runs **both** experts and decides from their outputs.
Chosen. The arbiter (b).
Cost of alternatives. An image-only router throws away the very evidence that best reveals the domain: how (un)confident each expert is. Running both experts costs one extra forward pass but yields the confidence, entropy and margin features that make routing accurate.
Consequence. Routing at ROC-AUC 0.973 - far above what an image-only prior achieves - for the price of one extra sub-second inference.

4.3.3. D-11 · XGBoost for the arbiter, over logistic regression or a neural net

Options. Logistic regression, a small MLP, or gradient-boosted trees.
Chosen. XGBoost.
Cost of alternatives. The decision boundary is non-linear in the 20 features (confidence interactions matter), which linear logistic regression cannot capture; an MLP needs more data and tuning and is harder to interpret. Trees also give us **TreeSHAP** per-prediction explanations for the "why" panel.
Consequence. 93-97% routing accuracy with a model small enough to re-implement as an eight-line JavaScript tree walk for the browser (verified identical to Python to 10^{-7}).

4.3.4. D-12 · Arbiter features = the experts' softmax statistics, not raw embeddings

Options. Feed the arbiter the concatenated 2560-D penultimate embeddings, or a compact set of confidence statistics (top-5 conf, entropy, margin, $P(\text{background})$, interactions).

Chosen. The 20 confidence statistics.

Cost of alternatives. Raw embeddings carry strictly more information (by the data-processing inequality) and are noted as a Future-Work upgrade, but they need PCA/an MLP and far more data to use without overfitting; the softmax statistics are tiny, interpretable, and already separable (the background feature alone is near-linearly separating).

Consequence. A robust arbiter trainable on 32k rows, with a clear upgrade path documented rather than prematurely taken.

4.3.5. D-13 · Cost-sensitive weighting $\alpha = 0.5$, threshold left at 0.5

Options. Balance the classes fully ($\alpha = 1$), or tune the decision threshold for Israeli recall.

Chosen. $\text{scale\_pos\_weight} = \left(\frac{n_{\text{neg}}}{n_{\text{pos}}}\right)^{0.5}$, threshold 0.5.

Cost of alternatives. We swept the threshold: overall accuracy is **flat** (83.75-83.85%) across 0.30-0.55, and full balancing floods the router with false Israeli routes. Lowering the threshold trades global for Israeli almost one-for-one (near zero-sum).

Consequence. We proved that threshold tuning only **redistributes** error; the real gain had to come from better information (the background feature), which is exactly what delivered it.

4.4. The Weight Channel

4.4.1. D-14 · A BLE scale, after rejecting vision-based weight estimation

Options. Estimate mass from the photo (monocular depth + volume, shadow geometry, a coin fiducial, Archimedes displacement as ground truth), measure depth with dedicated hardware (stereo-camera disparity or LiDAR / time-of-flight), read the scale's 7-segment display by OCR, or read the scale digitally over Bluetooth.

Chosen. Bluetooth Low Energy from a digital scale.

Cost of alternatives (measured). We built the vision pipeline: a coin fiducial for scale, MiDaS monocular depth for volume, shadow geometry for height, validated against Archimedes water displacement. Even when volume was accurate (an apple at 139.3 cm³, 7% versus displacement), the step **volume → mass** fails on unknown **density**, and the method only worked under controlled lighting. Dedicated depth hardware (stereo or LiDAR/ToF) was rejected at design time for the same density wall plus self-occlusion (a top-surface height field, not a closed volume), material failure on glossy/wet/dark food, and a cost/platform burden that breaks the any-device goal (see Section 8). OCR of the display needs a clear line of sight and still only reads what the scale shows.

Consequence. BLE reads true mass at the source, in any lighting, and became the reliable weight channel; the rejected experiments are a documented dead-end that justifies the choice.

4.4.2. D-15 · Software calibration through the origin, not a two-parameter line

Options. Trust the scale, apply a two-parameter linear correction, or a through-origin gain.
Chosen. corrected $= k \cdot$ raw, $k = 1.178$.
Cost of alternatives (measured). The scale read a systematic 16% low. A two-parameter fit on masses ≥ 162 g produced a spurious $+9.7$ g intercept that **over-read light items** (a 53 g portion read 64 g). Adding sub-160 g reference points showed the fault is a pure multiplicative **span error**, so the honest model passes through zero.
Consequence. Weight error fell from 16% to 1-3%, and the light-item bias vanished (verified in the calorie trials, where the weight channel averages 2.5%).

4.4.3. D-16 · Median filter over the raw decodes, not a sticky-high latch

Options. Latch the last non-zero high byte, or median-filter the last few decodes.
Chosen. A five-sample median.
Cost of alternatives. The latch turned a single transient overshoot into a **permanent** +256 g offset that never returned to zero (a removed-then-replaced load read double).
Consequence. The reading follows the scale down to zero and outvotes single-frame flicker.

4.5. Edge Deployment and the Web App

4.5.1. D-17 · Export to ONNX for a torch-free runtime

Options. Ship PyTorch/Ultralytics to the edge, or export to ONNX.
Chosen. ONNX.
Cost of alternatives. PyTorch + Ultralytics is a multi-gigabyte native dependency that cannot run in a browser at all and bloats a desktop build; ONNX is a framework-neutral graph any runtime can execute. The conversion is validated to **zero top-1 mismatches** and $\sim 10^{-3}$ max probability drift against PyTorch.
Consequence. A 0.8 GB torch-free desktop build and, crucially, the ability to run in the phone browser.

4.5.2. D-18 · Publish fp16, default to the CPU/WASM path, WebGPU opt-in

Options. fp32, fp16, or int8 weights; run on CPU (WASM) or GPU (WebGPU).
Chosen. fp16 files, WASM by default, WebGPU behind `?gpu=1`.
Cost of alternatives. fp32 doubles the download (100 MB vs 50 MB) for no gain - the CPU path ends up computing in fp32 either way, so fp32 files would only cost bandwidth. int8 risks accuracy on subtle food textures. On compute, WASM **widens fp16 to fp32** so the arithmetic stays clean and the answer makes the **same top-1 decision** as the desktop (only a 10^{-3} probability drift from the one-time fp16 rounding of the weights - decision-identical, not bit-identical), whereas WebGPU computes in fp16 and its rounding flipped a few borderline classes on some mobile GPUs. Because correctness outranks speed for a nutrition tool, WASM is the default.
Consequence. A small download, desktop-identical decisions on the default path, and an opt-in speed mode - documented so the trade-off is explicit.

4.5.3. D-19 · The arbiter as a JavaScript tree walk, not an ONNX tree operator

Options. Export the XGBoost model to ONNX and run it in ORT, or evaluate it in JavaScript.
Chosen. A hand-written tree walk in JS.
Cost of alternatives. ORT-Web's WebAssembly build does not ship the `ai.onnx.ml` operator domain, so an ONNX tree model **cannot execute in the browser**. A boosted ensemble is arithmetically trivial (sum of leaf values, then a sigmoid), so re-hosting it as eight lines of JS is exact, dependency-free and auditable.
Consequence. The full pipeline runs in the browser with the arbiter matching Python to 10^{-7}.

4.5.4. D-20 · A browser web app, not a native mobile app

Options. A native iOS/Android app, or a browser app.
Chosen. Browser (PWA-style static site).
Cost of alternatives. Native apps need per-platform toolchains, app-store review, and installs - friction that kills a demonstrator that must run on any handed-over phone. The browser reaches any device from a URL and keeps inference on-device.
Consequence. Zero-install edge AI, at the cost of Web Bluetooth being Android-only (documented; iPhone falls back to manual grams).

4.5.5. D-21 · Keys behind a Cloudflare Worker, never in the front-end

Options. Embed the USDA/Gemini keys in the page, or proxy through a serverless function.
Chosen. A single Cloudflare Worker holding the keys as encrypted secrets.
Cost of alternatives. The site is a **public** GitHub Pages repo; any embedded key is world-readable and would be abused.
Consequence. The front-end only knows a URL; the keys never ship, and the same Worker cleans the noisy USDA results.

4.6. Nutrition Fusion

4.6.1. D-22 · A three-tier nutrition lookup: local DB, then USDA, then Gemini

Options. A single source.
Chosen. Local JSON first, then USDA (filtered), then a Gemini fallback.
Cost of alternatives. A local DB alone cannot cover the long tail; USDA alone is noisy and misses some foods; a vision LLM alone is slow, costs money and needs connectivity. Tiering uses the cheap, exact source first and escalates only when needed.
Consequence. Instant answers for common foods, coverage for the rest, and honest offline behaviour when no tier is reachable.

4.6.2. D-23 · Cooking multipliers derived from USDA data, not invented

Options. Assume raw values, or apply hand-picked cooking factors.
Chosen. Multipliers computed from USDA raw-versus-cooked pairs (with sample count and 95% CI).
Cost of alternatives. Cooking concentrates or adds energy per gram; ignoring it biases fried foods badly, and invented factors are indefensible. Because the scale weighs the **cooked** food, per-100 g-cooked is the correct basis.
Consequence. Evidence-based factors (raw $\times$ 1.0 to deep-fried $\times$ 1.99) that we can cite.

4.6.3. D-24 · Validate calories by decomposing the error, not with a single number

Options. Report one end-to-end calorie error, or separate the channels.

Chosen. Split each trial into identity, weight-channel, and database error, and report absolute kcal alongside percentages.

Cost of alternatives. A single percentage hides **where** the error lives and is dominated by noise on near-zero-calorie foods (a 3-kcal miss on cucumber reads as 27%).

Consequence. We can state precisely that recognition is 100%, the weight channel is 2%, and the residual is the **food-database** entry choice (the irreducible term) - a diagnosis, not just a score.

4.6.4. D-25 · Ship the best-validation checkpoint, guarded by a regression gate for any fine-tune

Options. Ship the last epoch, or the best-validation epoch; allow ad-hoc fine-tuning on new data.

Chosen. Best-validation `best.pt`, and any fine-tune must pass an automatic clean-test regression gate.

Cost of alternatives (measured). We fine-tuned the Israeli model on tagged real-world captures four times; every run learned the new samples but **regressed the clean test set** (down to 23-66%) through catastrophic forgetting. The gate refused all four and the production weights stayed intact.

Consequence. A safe-by-construction retrain flywheel; the four failures are documented as the reason naive fine-tuning is the wrong tool, and a full retrain with an open-set class is the right one.

5. Verification and Validation

This chapter sets out the **plan** used to verify CalEyeZ; the measured outcomes are reported in Section 8. The guiding principle throughout is that every number must come from data the relevant model never trained on, and every claim must be reproducible from a script in the repository.

5.1. Test Objectives

The testing effort verifies four things: that each model meets its accuracy target on unseen data; that the router improves the system over the always-Global baseline without leakage; that the full pipeline runs within the latency budget on both GPU and CPU; and that the hardware and external-service paths (BLE weight, USDA lookup) behave correctly and fail gracefully.

5.2. Test Plan

Testing is organised in the usual hierarchy - unit, integration, and system - plus a field test on genuinely unseen photographs.

Unit level. Each model is evaluated on its own held-out **test** split. The metric is **top-1 accuracy** (fraction of images whose single most-probable class is correct) and **top-5 accuracy** (fraction whose correct class is among the five most probable):

$$\text{top-}k = \frac{1}{N}\sum_{n=1}^{N} \mathbb{1}[y_n \in \text{argtop}_k(p_n)]$$

The router is evaluated as a binary domain classifier; its headline metric is the area under the ROC curve (ROC-AUC), which is threshold-independent and therefore a fair measure of how well the routing signal separates the two domains.

Integration level. The two models plus the router are run together to produce the **routed system top-1 accuracy**, and compared against two references: the **always-Global baseline** (never route) and the **oracle** (a perfect router that picks the right expert whenever either is correct). The oracle is the system's accuracy ceiling and is what tells us whether errors are the router's fault or the models' fault.

Edge-parity test. The ONNX export must reproduce the PyTorch models exactly, so the edge build is trusted. The test runs both backends over the same images and records the fraction of top-1 disagreements and the maximum absolute probability difference; the pass condition is 0% top-1 mismatch.

Hardware and service tests. The BLE link is tested for correct weight decoding across the working range (including the carry-byte edge cases that earlier corrupted readings), for stable-reading detection, and for auto-reconnect after a dropped link. The nutrition path is tested for a correct USDA lookup, for the local-database fallback when offline, and for a graceful message on timeout.

Field test. A set of phone photographs taken outside the dataset (real plates, real lighting) is run through the deployed pipeline with the containing folder as ground truth. Because the sample is small, the accuracy is reported with a **Wilson score confidence interval** rather than a bare percentage. For $\hat{p} = x/n$ successes and $z = 1.96$ (95%):

$$\text{CI}_{95\%} = \frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\hat{p}\frac{1-\hat{p}}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}$$

5.3. Test Environment

Training and the GPU timing tests run on a desktop with an NVIDIA RTX 3060 Ti (8 GB), Python 3.10, PyTorch/Ultralytics. The edge-parity and CPU-latency tests run through ONNX Runtime on CPU only. Each test corresponds to a committed script: `train_general_model.py` and the Ultralytics validator for the models, `train_arbiter_xgb.py` for routing and the routed system metric, `onnx_export_and_check.py` for parity, `photo_tester.py` for the field test, and `scale_reader.py` for the BLE link.

5.4. Error Handling, Coverage and Documentation

Errors discovered during testing are fixed at the source and the test re-run; the most significant example is the BLE carry-byte bug described in Section 6.1, which a range-sweep test exposed and which was then fixed and re-verified. Coverage spans every functional requirement in Section 2: each model's accuracy, the routing decision, the end-to-end fused output, the two sensing channels, and the offline fallbacks. The results, tables and figures produced by these procedures are collected in Section 8 and the appendices.

6. Implementation

6.1. Hardware Design and Development

The hardware integration work concentrated on the **weight channel**, because the camera is a standard UVC webcam that needs no development. The scale is a consumer BLE unit (advertised name "SWAN"); getting a trustworthy weight out of it required reverse-engineering its Bluetooth protocol, since the manufacturer publishes no specification.

The reverse-engineering method was deliberately empirical. Using the **nRF Connect** phone app we scanned for BLE advertisers; the scale exposes only a generic name, so we identified it by **signal strength** - holding the phone against the scale, the correct device is the one whose RSSI jumps toward 0 dBm while every other advertiser stays weak. We then connected, listed the GATT services, and subscribed to the **notify characteristic** (`0000ffb2-....-9b34fb`), which streams a fresh 8-byte frame on every change. Treating the scale as a black box, we placed **known masses** (100 g, 200 g, then crossing 255 g), recorded the packet each time, and wrote the readings down side by side; the bytes that moved with weight, and how they moved, revealed the encoding: the weight in grams is little-endian, with the low byte in `packet[4]` and the count of 256-gram carries spread across `packet[5]` and `packet[3]`. Crossing 255 g is what exposed the high byte. The same side-by-side packet log also revealed the frame's **integrity fields**: every frame starts with the constant header `0xAC`, and the last byte always equals the low 8 bits of the sum of bytes 2 through 6 - a simple additive **checksum**, confirmed by predicting byte 7 from the payload on dozens of captured frames. The driver validates both on every notification (`len >= 8`, `pkt[0] == 0xAC`, `sum(pkt[2:7]) & 0xFF == pkt[7]`) and silently drops any frame that fails, so a corrupted radio packet can never become a weight. An early version masked the low bit of the high byte, which silently dropped 256 g on every odd multiple of 256 (so 272 g read as 16 g); removing that mask fixed it. The decode and a small median filter that rejects single-frame flicker and overshoot are shown below:

```
def notification_handler(sender, data):
    pkt = list(data)
    if len(pkt) < 8: return
    low  = pkt[4]
    high = pkt[5] | pkt[3]          # carries; do NOT mask bit0 (drops 256 g)
    raw_buffer.append(low + high * 256)
    srt = sorted(raw_buffer)
    weight = srt[len(srt) // 2]     # median of last few decodes -> rejects
flicker
    stability_buffer.append(weight) # STABLE once last 10 samples vary <= 2 g
```

The choice of a **median** rather than a mean is the classical robust-statistics argument: the mean of the last five samples is dragged by a single outlier (one flicker frame of 0 g under a 500 g load pulls a 5-sample mean down by 100 g), whereas the median is unchanged by up to two arbitrary outliers out of five - its **breakdown point** is 50%. Since the BLE stream's failure mode is exactly occasional single-frame flicker and overshoot, not Gaussian noise, the median is the right filter; a mean would smooth genuine noise better but is defenceless against the spikes that actually occur.

Every notification passes through the same fixed **weight-calculation state machine** before it is shown, so the value that is multiplied into a calorie figure is never a single raw frame:

Stage	Computation	Output
Validate	`len >= 8`, header `== 0xAC`, `sum(b[2..6]) & 0xFF == b[7]`	pass / **drop**
Decode	`low = b[4]`; `high = b[5] | b[3]`; `raw = low + high*256`	raw grams
Median	median of the last 5 `raw` values	flicker-free
Calibrate	`g = round(1.178 * median)` (through-origin span fix)	true grams
Stability	max − min of last 10 `g` $\leq$ 2 g?	STABLE / MOVING
Emit	`current_weight = g`	used for calories

A reading is declared **stable** once the last ten samples differ by no more than 2 g, which is the value the GUI latches when the user clicks Analyze. The link runs on a background thread with an auto-reconnect loop so a dropped connection recovers on its own.

The driver is structured as a **finite state machine**, because a live sensor link is never simply "connected or not" - it drops, flickers and goes silent. Explicit states, a **staleness watchdog** (no packet for 4 s triggers an automatic reconnect) and a **stability gate** are what make the reading trustworthy enough to multiply into a calorie figure; when the machine cannot reach `STREAMING`, the manual-grams entry is the fallback.

State	Event	Action	Next
DISCONNECTED	user connects	start BLE scan	SCANNING
SCANNING	"SWAN" / strongest RSSI	open GATT connection	CONNECTING
CONNECTING	GATT connected	enable notify on `ffb2`	SUBSCRIBING
SUBSCRIBING	notify enabled	start packet loop	STREAMING (MOVING)
STREAMING	valid packet	decode, push to median	MOVING / STABLE
STREAMING	bad header/len/checksum	drop the frame	STREAMING
MOVING	last 10 vary $\leq$ 2 g	latch settled reading	STABLE
STABLE	a sample varies $>$ 2 g	reading moving again	MOVING
STREAMING	no packet $>$ 4 s	tear down, re-scan	STALE $\rightarrow$ CONNECTING
any	GATT disconnect	reset weight to 0	DISCONNECTED

6.2. Software Development

The software was built in four stages, each a committed, re-runnable script.

Dataset preparation. The raw collection of 142 folders was cleaned into a leakage-free 132-class set: visually identical sub-classes were merged, mislabelled and disallowed classes removed, and exact plus perceptual de-duplication applied before a fresh 70/20/10 split. The de-duplication is what removed the train/test leakage that had previously inflated the old model's test score; after the rebuild the validation and test accuracies agree (88.16% vs 88.18%), which is the proof that the splits are clean.

Model training. Both classifiers are YOLO11l-cls trained by transfer learning, configured exactly as in Section 3.3 - AdamW, cosine learning-rate decay, dropout, and heavy augmentation including a custom Albumentations blur callback to mimic classroom motion blur. The 320 px input at batch 16 was chosen to fill the 8 GB VRAM budget without overflowing it.

Router. Both trained models are run over the val and test images to build a feature table (one row per image), and XGBoost is trained on the val rows and scored on the test rows. The generator deliberately **skips** the Israeli model's background class as a routing row and instead exposes it as the `i_p_background` feature, which became the single most informative input.

Edge export. Finally both models are exported to ONNX so the system can run on a CPU-only device without PyTorch. The key correctness detail is that the torch-free preprocessing must replicate Ultralytics' classification transform exactly - resize shortest edge to the model size, centre-crop, scale to $[0, 1]$ with no ImageNet normalisation - or the probabilities drift:

```
def preprocess(bgr, size):                # torch-free replica of ultralytics
transform
    rgb = cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)
    h, w = rgb.shape[:2]
    nh, nw = (size, round(w*size/h)) if h <= w else (round(h*size/w), size)
    r = cv2.resize(rgb, (nw, nh))
    y0, x0 = (nh - size)//2, (nw - size)//2
    c = r[y0:y0+size, x0:x0+size]
    x = c.astype(np.float32) / 255.0      # NO imagenet mean/std
    return np.ascontiguousarray(np.transpose(x, (2,0,1))[None])
```

6.3. Integration of Hardware and Software

The pieces are tied together by the Fusion Logic Controller in the demo application. On an Analyze event it captures the camera frame and the latched weight, runs both experts on the frame, builds the 20-feature row, asks the router for $P(\text{israeli})$, selects the expert, and resolves nutrition. The selection also applies guard conditions - a confidence gate, an "arbiter undecided" band around $P = 0.5$, and an Israeli-model **abstain** when the chosen label is `background` - so a weak result can be flagged rather than reported as fact:

```
p_israeli = float(arbiter.predict_proba(X)[0, 1])
route_israeli = p_israeli >= 0.5
chosen = israeli if route_israeli else global_
israeli_abstain = route_israeli and chosen["label"] == "background"
confident = (chosen["conf"][0] >= gate) and not both_unsure \
            and not arbiter_unsure and not israeli_abstain
```

The same code path works unchanged whether the experts are the PyTorch models or the ONNX backend, because both return an identical feature dictionary; the backend is chosen by an environment variable (and automatically if PyTorch is absent).

6.4. Deployment and Installation

For deployment to a low-power machine the application is packaged with PyInstaller into a standalone executable that bundles ONNX Runtime and the two ONNX models but **excludes** PyTorch and Ultralytics, giving a small, torch-free build. On a CPU this runs an analysis in roughly 0.35 s, versus 1–3 s for the torch CPU path. The packaged executable ships with **no** API keys; the user supplies a USDA key through an environment variable or a key file beside the executable, and an empty key simply yields a blank nutrition panel rather than an error.

6.5. Configuration and Setup

Setup is minimal: pair the SWAN scale once (the app scans and connects by name), connect the webcam, and optionally provide a USDA key. A camera selector (a button and a hotkey) cycles through working camera indices so a machine with both an internal and a USB camera can switch between them. No **per-user** calibration is required: the span correction of Section 8.5 ($k = 1.178$) is a device constant baked into the software, so the user never calibrates anything - place the food and read the corrected weight.

6.6. Bill of Materials

Item	**Specification**	**Status**
Digital BLE scale	Consumer scale, BLE 4.0+, strain-gauge load cell, custom GATT notify profile (device “SWAN”)	Purchased
USB webcam	1080p RGB, USB 2.0/3.0, UVC	Existing
Desktop workstation	NVIDIA RTX 3060 Ti (8 GB) for training/inference	Existing
Edge device (optional)	CPU-only PC, 8 GB RAM, for the ONNX build	Existing
Software stack	Python 3.10, PyTorch/Ultralytics, XGBoost, ONNX Runtime, OpenCV, Bleak, CustomTkinter	Open source

7. The Web Application: Edge AI in the Browser

The desktop demo proves the system works; the web application proves it can **ship**. It takes the exact same pipeline - two YOLO11l-cls experts, the XGBoost arbiter, the BLE weight, the USDA fusion - and runs it **inside an unmodified phone browser**, with no installation, no app store, and no server performing the recognition. A user opens a URL, points the camera at a plate, and the food is identified on the device itself. This chapter explains how a stack built in Python and PyTorch was made to run in JavaScript on a mobile CPU, and why each conversion was necessary.

The live app, the diagnostic console, and the latency benchmark are public: caleyez-web, engineering.html, and bench.html.

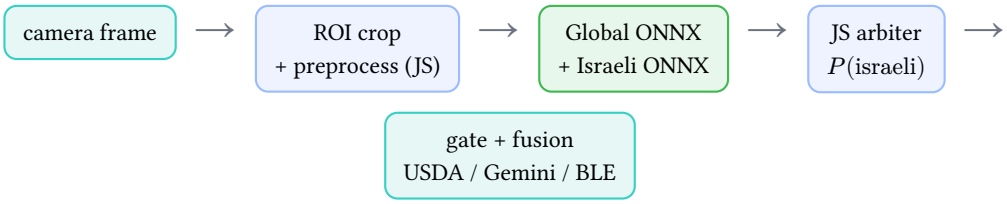
7.1. Why a Browser, and What "Edge AI" Means Here

Three properties motivated the browser target. **Zero install**: a web page reaches any phone regardless of operating system, which matters for a demonstration that has to work on whatever device is handed to it. **Privacy and offline operation**: because the neural networks execute locally, the captured image never leaves the phone for recognition, and once the model files are cached the identification works with the network switched off. **Honesty**: the app times the inference and prints `N ms on-device edge` on every result, so the claim "this ran on your phone, not in the cloud" is visible rather than asserted.

"Edge AI" is the precise term for this: the inference happens at the edge of the network, on the end-user device, rather than on a central server. The engineering problem is that the edge device is a battery-powered phone CPU, perhaps two orders of magnitude weaker than the RTX 3060 Ti that trained the models. Everything that follows - the ONNX conversion, the fp16 weights, the deliberately small input sizes, the JavaScript arbiter - exists to fit a training-grade pipeline into that budget without changing a single prediction.

7.2. Architecture Overview

The browser app mirrors the desktop Fusion Logic Controller of Section 6.3 stage for stage:



Only three things are genuinely new relative to the desktop build, and they are the subject of this chapter: (1) the two classifiers are executed by **ONNX Runtime Web** instead of PyTorch; (2) the XGBoost arbiter is evaluated by a hand-written **JavaScript tree walker** instead of the XGBoost library; and (3) nutrition and the Gemini fallback are served by a **Cloudflare Worker** so that no API key ships in the public front-end. The recognition math is otherwise identical, and that identity is verified numerically rather than assumed.

7.3. The On-Device Runtime: ONNX Runtime Web

The single JavaScript dependency for inference is `onnxruntime-web` (ORT-Web), loaded from a CDN. ORT-Web ships the ONNX Runtime engine compiled to **WebAssembly** (WASM)[11]

- a low-level, near-native

bytecode that every modern browser executes - plus an optional **WebGPU** backend. Configuration is three lines:

```
ort.env.wasm.wasmPaths = "https://cdn.jsdelivr.net/npm/onnxruntime-web/dist/";
ort.env.wasm.numThreads = 1;                          // no SharedArrayBuffer /
COOP-COEP needed
const ep = wantGpu ? ['webgpu','wasm'] : ['wasm']; // WASM is the default; ?
gpu=1 opts into WebGPU
```

Single-threaded WASM is a deliberate choice. Multi-threaded ORT-Web needs `SharedArrayBuffer`, which browsers gate behind cross-origin isolation (COOP/COEP HTTP headers) that a plain GitHub Pages static host cannot set. Rather than complicate hosting, the app runs one WASM thread; the inference still completes in well under a second because the models are sized for it. The two execution providers - WASM on the **CPU** and WebGPU on the **GPU** - differ in a way that turns out to be the central numerical decision of the whole port, covered in Section 7.6.

7.4. Loading the Models into RAM

A model file must travel three stages to become callable: **network to bytes**, **bytes to session**, and **session to compiled kernels**.

Network to bytes. Each `.onnx` file is streamed with `fetch` and reassembled into an `ArrayBuffer`, reading the `content-length` header so the splash screen can show a real download bar (the first visit fetches roughly 50 MB, which is a blank wait otherwise):

```
async function mk(url, ep, onProg){
  return ort.InferenceSession.create(await fetchBuf(url, onProg),
                                     { executionProviders: ep });
}
G = await mk(MODEL_BASE + 'global_fp16.onnx',  ep, prog(0));   // 0-50% of the
bar
I = await mk(MODEL_BASE + 'israeli_fp16.onnx', ep, prog(50));  // 50-100%
```

The models are hosted cross-origin in the main `CalEyeZ` repository under `/webmodels/` and fetched over CORS, so the app repository stays a few kilobytes and deploys in seconds. After the first load the browser HTTP-caches the files, so every later launch is instant and fully offline.

Bytes to session. `InferenceSession.create` parses the ONNX graph and allocates the weight tensors in the WASM heap. This is where fp16 becomes fp32: on the WASM path ORT upcasts each 16-bit weight to a 32-bit float as it loads, so the working set in RAM is the full-precision model (a few hundred megabytes touched during inference). The app therefore targets phones with $\geq$ 2 GB RAM.

[11] A portable, near-native compilation target now standard in every major browser. The GPU path uses the W3C *WebGPU* specification, the browser's successor to WebGL for general-purpose GPU compute.

Session to compiled kernels. The first `run()` on a fresh session triggers kernel compilation and memory-plan construction, which would make the first photo mysteriously slow. A `warmup()` runs both models once on a grey dummy frame during the splash "calibrating…" step, so the user's first real capture is already fast.

One subtlety specific to WASM/WebGPU memory management is worth stating, because it caused a real crash. ORT-Web tensors are backed by WASM (or GPU) memory that the JavaScript garbage collector does **not** reclaim. Every input tensor and every `run()` output map must be explicitly disposed, or the heap grows on each capture until the GPU path crashes:

```
dispose(tg, ti, go, io);   // free WASM/GPU tensor memory the JS GC will not
collect
```

7.5. From YOLO to ONNX, and Why

PyTorch and Ultralytics cannot run in a browser - they are large native Python packages. ONNX (Open Neural Network Exchange) solves this: it is a **framework-neutral file format** for a computation graph. Exporting each trained `.pt` to `.onnx` decouples the model from PyTorch, so any ONNX runtime

- including the WebAssembly one - can execute it. The export is one call, with the classification

softmax already fused into the graph so its output is class probabilities directly:

```
m = YOLO("runs/general_model_flattened/weights/best.pt")
m.export(format="onnx", imgsz=320, opset=12, simplify=True, dynamic=False)
```

`opset=12` pins a widely supported operator set; `simplify=True` runs a graph-simplifier that folds constants and fuses redundant nodes (fewer kernel dispatches at run time); `dynamic=False` fixes the input shape to $1 \times 3 \times 320 \times 320$, which lets the runtime pre-plan memory exactly.

The parity guarantee. A format conversion is only useful if the answer does not change. The export is validated by running both the PyTorch model and the ONNX model on the same 25 field images and measuring the largest per-class probability difference and any top-1 disagreement:

```
pt_probs = m.predict(bgr, imgsz=size)[0].probs.data.cpu().numpy()
onnx_out = sess.run(None, {inp: preprocess(bgr, size)})[0][0]
maxdiff  = np.abs(pt_probs - onnx_out).max()      # required < 0.02, observed
~1e-3
mismatch = int(pt_probs.argmax() != onnx_out.argmax())   # required 0
```

Both models pass with zero top-1 mismatches and a maximum probability drift on the order of 10^{-3}, which is pure floating-point noise. The one thing that **will** break parity is the pixel pre-processing: the ONNX side must replicate Ultralytics' classification transform exactly - resize the shortest edge to the model size, centre-crop, scale to $[0, 1]$, and crucially apply **no ImageNet mean/std normalisation** (YOLO11-cls is trained on raw $[0, 1]$ pixels). The same transform is what the JavaScript front-end implements, discussed in Section 7.8.

7.6. fp16 vs fp32: the Mathematics of the Hardware Limit

This is where the port earns its performance. The models are published in **fp16** (half precision), and understanding why requires three numbers: memory, bandwidth, and rounding error.

Memory. A single-precision weight (fp32) occupies 4 bytes; a half-precision weight (fp16) occupies 2 bytes. A model with P parameters is therefore $4P$ bytes as fp32 and $2P$ as fp16. For the Global model this is the difference between roughly 50 MB and **25 MB** on disk and over the network; across both models the download halves from about 100 MB to about 50 MB. On a phone on conference wifi, that halving is the difference between a usable first load and an abandoned one.

Bandwidth - why the phone works less hard. Convolutional inference on a CPU is frequently **memory-bound**, not compute-bound: the processor spends more time waiting for weights and activations to arrive from RAM than doing arithmetic. The relevant quantity is **arithmetic intensity**, the ratio of floating-point operations to bytes moved. By the roofline model[12] the attainable throughput is

$$\text{throughput} = \min(\text{peak compute}, \text{memory bandwidth} \times \text{arithmetic intensity}).$$

Halving the bytes per weight **doubles** the arithmetic intensity, which lifts the bandwidth-bound ceiling and lets the same CPU finish sooner for the identical arithmetic. This is the precise sense in which fp16 lowers hardware usage: not fewer multiplications, but half the data traffic feeding them.

Separately, the **input resolution** controls the multiplication count itself. A convolution producing an $H \times W$ output map with C_{out} channels from C_{in} input channels through a $k \times k$ kernel costs

$$\text{MACs} = H \cdot W \cdot C_{\text{out}} \cdot C_{\text{in}} \cdot k^2$$

multiply-accumulates. Spatial size enters as $H \cdot W$, so cost grows with the **square** of the input edge. Choosing 320^2 for the Global model and 224^2 for the Israeli model - rather than the full camera resolution - is what keeps the MAC count low enough to finish on a phone CPU in a fraction of a second. fp16 and small inputs attack the two different bottlenecks (bytes moved and operations performed) at once.

Rounding - why fp16 is not the default. fp16 has a 10-bit mantissa, giving roughly $2^{-11} \approx 5 \times 10^{-4}$ relative spacing between representable values - about three to four significant decimal digits. Rounding every weight and activation to fp16 injects a small relative error at each layer; accumulated through a deep network it can flip a **borderline** prediction whose top-1/top-2 margin $p_{(1)} - p_{(2)}$ is tiny. fp32 has a 23-bit mantissa ($\approx 6 \times 10^{-8}$ spacing), so its arithmetic is effectively exact for our purposes.

The two backends resolve this differently, and that is the whole decision:

	WASM (CPU) - default	**WebGPU (GPU) - `?gpu=1`**
Weights on disk	fp16 (small download)	fp16 (small download)
Compute precision	**upcast to fp32**	native **fp16**
Accuracy	same **top-1 decision** as desktop ($\sim 10^{-3}$ prob drift)	can flip borderline classes on some mobile GPUs
Speed	set by CPU, 0.6 s on a flagship	faster fp16 matrix maths on the GPU

[12] S. Williams, A. Waterman, and D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65-76, 2009. The model bounds attainable performance by the minimum of a compute ceiling and a bandwidth ceiling that scales with arithmetic intensity.

So the phone downloads the small fp16 files either way. On the default CPU/WASM path each weight is widened back to fp32 before any arithmetic. It is worth being precise about what this does and does **not** do: widening **cannot recover the bits that fp16 rounding threw away** - a widened fp16 value is the **same** rounded number, just written with trailing zeros - so no information is invented. What it buys is clean **arithmetic**: the millions of multiply-adds run in fp32 and are not re-rounded to 16 bits at every step, which is exactly where fp16 **compute** loses accuracy. The consequence is recognition that makes the **identical top-1 decision** as the desktop model, with only a 10^{-3} probability drift left over from the one-time fp16 rounding of the stored weights at export (the same drift measured in the ONNX parity test, Section 7.5). It is decision-identical, not bit-identical. The cost is that the maths runs on the CPU, whose clock speed sets the latency directly (about 0.6 s on a recent flagship, proportionally slower on a budget chip). WebGPU keeps the weights in fp16 and computes on the GPU - faster, but the live rounding degraded a few classes on some mobile GPUs during testing. Because correctness outranks speed for a nutrition tool, **WASM/fp32-accurate is the default** and WebGPU is an explicit opt-in.

To make concrete **why** widening helps when it adds no information, picture fp16 as a calculator that shows **4 digits** and fp32 as one that shows **8**. The weights are 4-digit-accurate either way, so padding zeros changes nothing about them - but it changes the **running total** a layer builds from thousands of multiply-adds. On the 4-digit calculator $1000 + 0.04 = 1000$: the small term is below its precision and is **swamped**, so a hundred such additions still read 1000 when the truth is 1004. The 8-digit calculator keeps $1000.0400 \rightarrow 1004.0000$. A convolution sums thousands of such terms, so fp16 **arithmetic** silently drops small activations that can tip a borderline class; widening to fp32 loads the same 4-digit weight into an 8-digit register so every subsequent product and sum has room. The desktop computes with the same 8-digit (fp32) arithmetic, which is why the browser reproduces its **decision** - the only residual difference is 4-digit versus 8-digit **inputs** (the $\sim 10^{-3}$ export rounding), which does not accumulate. Computing directly in fp16 (WebGPU) instead runs every step on the 4-digit calculator, compounding the rounding thousands of times - the mechanism behind the flipped classes above.

The same loss is visible in a **single multiply**, the atom of the computation, and it is really about the **mantissa** - the significant-digit part of a float (a number is stored as sign $\times$ mantissa $\times 2^{\text{exponent}}$, and the mantissa's bit-count sets the precision: 10 bits in fp16, 23 in fp32). Multiplying two values each good to 4 significant digits produces a product that needs 7 digits to write exactly - for example $1.234 \times 5.678 = 7.006652$. fp16 must round that back to 4 digits (7.007), discarding the tail; fp32's 23-bit mantissa keeps all seven. In bit terms, two 10-bit mantissas multiply into an up-to-20-bit result: fp32 has room to hold it, fp16 does not and rounds at every one of the layer's multiply-accumulates. This is exactly the "room to grow" intuition - the product of two 16-bit numbers wants roughly 32 bits, and only fp32 keeps them. Widening the stored fp16 weights to fp32 before the arithmetic gives each intermediate product that room, which is the precise sense in which the model **computes at higher precision than it stores**.

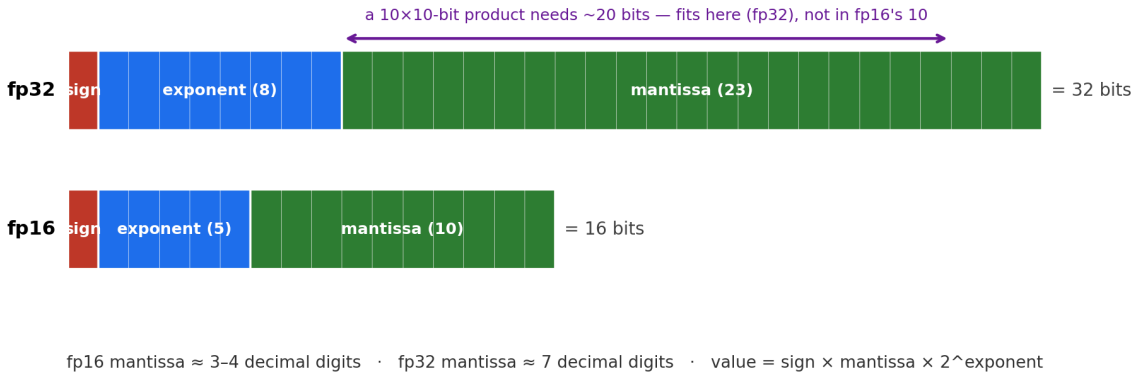


Figure 7: The bit layout of both formats. A float is sign $\times$ mantissa $\times$ 2^{exponent}; the **mantissa** holds the significant digits and its width sets the precision - 10 bits (4 decimal digits) in fp16 versus 23 bits (7 digits) in fp32. The exponent handles magnitude, so fp16 is short on **digits**, not range. Multiplying two 10-bit mantissas yields an up-to-20-bit result that fits in fp32′s 23-bit mantissa but not fp16′s 10 - the "room to grow" that lets fp32 accumulate without shedding precision.

Why the residual gap is only about 10^{-3}. When the CNN path does match the desktop only "in decision," it is worth quantifying how small the leftover disagreement is, because the number falls straight out of the mantissa width. The chain has three links. **First,** fp16 rounding perturbs each weight by at most half a mantissa step: near 1.0 consecutive fp16 values are $2^{-10} \approx 0.98 \times 10^{-3}$ apart, so rounding to the nearest is off by at most $2^{-11} \approx 4.9 \times 10^{-4}$ in relative terms - every weight becomes $w(1 \pm 5 \times 10^{-4})$. **Second,** a logit $z = \sum_i w_i x_i$ inherits that relative error: in the worst case (all errors aligned) $|\Delta z| \,/\, |z| \leq 5 \times 10^{-4}$, and because the errors are really random in sign they partly cancel, so the typical drift is smaller; across depth it compounds to at most a few $\times\, 10^{-3}$. **Third,** the softmax passes a logit change through almost one-for-one. A concrete instance: take a top logit $z_1 = 8.000$ and a runner-up $z_2 = 6.000$ (a modest margin of 2). The two-class softmax is $p_1 = 1/\left(1 + e^{-(z_1 - z_2)}\right)$, so $p_1 = 1/(1 + e^{-2.000}) = 0.88080$ in fp32; if fp16 rounding shifts the margin to 2.004, $p_1 = 1/(1 + e^{-2.004}) = 0.88121$ - a change of $\Delta p \approx 4 \times 10^{-4}$. The **maximum** over every class and every held-out row, measured in the ONNX parity test (Section 7.5), is $\approx 10^{-3}$. The decision only flips if the top two probabilities sit **within that 10^{-3}** of each other; real top-1/top-2 margins are on the order of tenths (e.g. 0.88 versus 0.12), so a 10^{-3} wiggle never crosses them - which is exactly why the exhaustive parity check found **zero** top-1 mismatches. The drift is real, bounded by the mantissa width, and an order of magnitude below anything that could change an answer.

Note: One weight, followed end to end - the whole story on a single number. Take a trained weight $w = 0.71341827$ (fp32).

1. **Cut to fp16 (at export).** It is stored as 0.7134; the low digits …1827 are dropped for good. This one rounding is the **only** precision ever lost.
2. **Widened in memory (at load).** To compute, 0.7134 is written into a 32-bit slot as 0.71340000 - the freed low bits are filled with **zeros**. No information is added; it is still 0.7134.
3. **The arithmetic fills the zeros.** Multiplying by an activation $a = 0.8207$ gives $0.7134 \times 0.8207 = 0.58548738$ - a product whose new low digits …8738 land exactly in the slots that were zero. In fp16 they are re-rounded away (0.5855); in fp32 they **survive**.
4. **The numbers drive the logit.** A neuron sums thousands of such products. Keeping those tails (fp32) instead of chopping them at every step (fp16) is what makes the summed logit match the full model - here 8.004 against the full-fp32 model's 8.000.
5. **Same decision; the drift is blown away.** Against a runner-up logit of 6.000, the softmax gives $P = 1/(1 + e^{-2.004}) = 0.88121$ versus the full model's 0.88080 - a difference of 4×10^{-4}. The winner leads by tenths (0.88 vs 0.12), so a 10^{-3} nudge cannot cross the gap. The light fp16-shipped model reaches the **identical decision** as the heavy fp32 model; only the third decimal of the probability differs, and that is below anything that decides an answer.

Note: The design pattern is worth stating plainly: **quantise for transport, compute at full precision**. The 16-bit format makes the model small and cheap to move; widening to 32 bits before the arithmetic adds **no** precision back to the weights themselves - it only stops the **computation** from shedding any more, which is where fp16 maths actually goes wrong.[13]

7.7. Converting the Arbiter: XGBoost Tree to JavaScript

The arbiter is a gradient-boosted tree ensemble, and it could **not** be exported to ONNX like the CNNs were. ONNX does define a tree operator (`ai.onnx.ml.TreeEnsembleClassifier`), but the WebAssembly build of ORT-Web does **not** ship the `ai.onnx.ml` operator domain, so a tree model simply will not execute in the browser through ONNX Runtime. The arbiter had to be evaluated another way.

The solution exploits the fact that a gradient-boosted ensemble is **arithmetically trivial to evaluate** - all the intelligence is in the training, not the inference.

The idea in plain terms. The arbiter is **400 tiny yes/no flowcharts** (decision trees). For one meal, each flowchart asks a couple of questions about the two experts' confidence numbers (for example "is the Israeli model's not-mine score below 0.00005?") and lands on a **leaf holding one small vote**. Add up all 400 votes plus a fixed **starting number** (the base) to get a single score in **log-odds**, then a **sigmoid** turns that score into a probability between 0 and 1 - the chance the meal is Israeli. Above 0.5, route to the Israeli expert. Formally, the model is a plain additive sum over $K = 400$ trees:

$$\text{score}(x) = \text{base} + \sum_{k=1}^{K} f_k(x), \quad f_k(x) = w_{q_k(x)}, \quad P(\text{israeli} \mid x) = \sigma(\text{score}(x)) = \frac{1}{1 + e^{-\text{score}(x)}}$$

[13] We deliberately stop at fp16-for-storage and avoid int8, whose coarser steps can flip the subtle textures that separate look-alike foods.

where each tree f_k routes the feature vector x from its root to a leaf $q_{k(x)}$ and contributes that leaf's scalar vote w. Every internal node is a single comparison "feature $j <$ threshold?". Evaluating the whole model is therefore: walk each tree by comparisons, add up the leaf votes and the base, and squash with the logistic function - a few hundred comparisons, sub-microsecond.

The export - three mechanical steps (script: `scripts/arbiter/export_arbiter_trees.py`). Converting the trained booster to the browser file is pure copying, not retraining:

1. **Dump the trees.** XGBoost's own `booster.get_dump("json")` writes all 400 trees as JSON (each node's feature, threshold, children, and leaf values).
2. **Flatten each node** into a tiny array: an internal node becomes `[feature_index, threshold, yes, no]` and a leaf becomes `[value]`.
3. **Compute the base.** The starting number is the log-odds of the overall Israeli rate, base $= \ln(p/(1-p))$ with $p = 0.2176$ (Israeli images are the minority of training rows), giving -1.2795 - negative because global images dominate.

The result is `arbiter_trees.json` with two keys: `base` and `trees` (the 400 flat trees). Nothing about XGBoost survives into the browser except these numbers - **the file is the model**. The entire inference engine is then eight lines of JavaScript:

```
function arbiterP(f){                                    // f = the 20 features
  let m = ARB.base;                                      // start at base log-odds
(-1.2795)
  for(const tree of ARB.trees){
    let n = tree["0"];                                   // start at the root
    while(n.length > 1){                                 // length 4 = internal, 1
= leaf
      n = tree[ Math.fround(f[n[0]]) < n[1] ? n[2] : n[3] ];  // [feat, thresh,
yes, no]
    }
    m += n[0];                                            // add this tree's vote
(leaf value)
  }
  return 1 / (1 + Math.exp(-m));                          // sigma(score) =
P(israeli)
}
```

The one subtlety is `Math.fround`. XGBoost stores thresholds and compares feature values in **32-bit** floats, so the JavaScript casts each feature to fp32 before the `< threshold` test; without it, a handful of values sitting exactly on a threshold boundary would branch the wrong way in fp64 and flip $\approx 0.08\%$ of routing decisions. With it, the walk reproduces XGBoost's prediction path node for node - a **lossless** re-implementation, not an approximation.

The validation - proving it acts the same. Equivalence is proven, not assumed, and the export script performs the check on every run. The **entire** held-out feature table - all **32,136** rows the arbiter is scored on - is pushed through both `booster.predict_proba` in Python and the JavaScript walk, and the two probability vectors compared element-wise. The maximum absolute disagreement is 3.8×10^{-7} (pure 32-bit rounding in the 400-term sum), and the number of differing routing **decisions** (the $P \geq 0.5$ comparison) is **zero**. This is the same parity discipline applied to the CNNs' ONNX export (Section 7.5): every model that crosses a language or runtime boundary carries a numerical-equivalence proof with it, so the browser's routing decision is not "similar to" the desktop's - it is the same computation.

The 20-element feature vector `f` is assembled in the identical training order - both experts' top-5 confidences, entropy $H(p) = -\sum_i p_i \log p_i$, margin $p_{(1)} - p_{(2)}$, the Israeli `background` probability, and the four interaction features - directly from the two ONNX outputs, so the arbiter in the browser sees exactly what the arbiter in training saw.

> **Note:** Why not run the whole arbiter as a tiny neural network, or in ONNX at all? Because the honest engineering answer is that the model is a decision tree, and a decision tree is **ten comparisons you can read**. Re-hosting it as JavaScript keeps it exact, dependency-free, and auditable, and sidesteps the missing `ai.onnx.ml` kernels entirely.

7.8. The Pixel Pipeline in JavaScript

The parity guarantee of Section 7.5 depends on the browser feeding the ONNX models the **same tensor** the Python pre-processing would. This is classical image processing, done in code before any network runs: take the central 60% region of interest, resample it to the model's square input, drop the alpha channel, scale to $[0, 1]$ by dividing by 255, and reorder the pixels from interleaved `HWC` (height-width-channel, the browser's canvas layout) to planar `CHW` (channel-height-width, the tensor layout ONNX expects):

```
function tensorFromCanvas(cv, sz){
  const d = cv.getContext('2d').getImageData(0,0,sz,sz).data;   // RGBA, 0..255,
HWC
  const n = sz*sz, out = new Float32Array(3*n);
  for(let i=0;i<n;i++){                                          // de-interleave
+ /255
    out[i]     = d[i*4]   / 255;   // R plane
    out[i+n]   = d[i*4+1] / 255;   // G plane
    out[i+2*n] = d[i*4+2] / 255;   // B plane (alpha d[i*4+3] dropped)
  }
  return new ort.Tensor('float32', out, [1,3,sz,sz]);
}
```

No ImageNet normalisation is applied, matching the training transform exactly. The Global model gets its 320^2 crop and the Israeli model its 224^2 crop from the same frame, and both are run in the same `analyze()` call whose wall-clock time becomes the `on-device edge` latency chip.

7.9. The Nutrition Backend: a Cloudflare Worker

Recognition is fully local, but nutrition lookup and the Gemini fallback need cloud services - and those need API keys that must never appear in a public front-end. The solution is a single **Cloudflare Worker** (edge serverless function) that holds the keys as encrypted secrets and is routed by HTTP method:

- **`GET /?q=<food>`** queries USDA FoodData Central and returns per-100 g `{kcal, pro, carb, fat}`. A **smart filter** cleans the notoriously noisy USDA results: it rejects junk descriptors (juice, dried, sauce, ...), scores candidates by primary-name and token overlap, treats cooking words as stop-words, and penalises ALL-CAPS branded entries. `?debug=1` returns the ranked candidate list, which the engineering console displays.
- **`POST {image}`** forwards the ROI to Gemini vision (`gemini-flash-latest`) for the "unsure" fallback, returning a food label.

The front-end never sees a key; it only knows the Worker URL. The **offline story** is handled explicitly: recognition works with no network at all, and when the on-device system is unsure but the device is offline, the app says so honestly ("no connection") rather than firing a confident guess or silently failing.

7.10. Evidence-Based Cooking Multipliers

A raw-weight calorie figure is wrong for cooked food, because cooking concentrates or adds energy per gram. Rather than invent factors, they are **derived from data**: `cooking_multipliers.json` is generated by querying USDA raw-versus-cooked pairs for the same food and taking the per-100 g cooked/raw kcal ratio as the multiplier, aggregated with a sample count and a 95% confidence interval. Because the BLE scale weighs the **cooked** food on the plate, per-100 g-cooked is the correct basis. The resulting factors are $\times$ 1.0 raw, $\times$ 1.04 boiled, $\times$ 1.17 grilled, $\times$ 1.47 fried, and $\times$ 1.99 deep-fried, and the final calorie estimate is

$$\mathrm{kcal} = \mathrm{kcal}_{\text{per 100 g, raw}} \times \frac{\mathrm{weight}_g}{100} \times \mathrm{multiplier}_{\mathrm{method}}.$$

7.11. Weight in the Browser: Web Bluetooth

The same reverse-engineered SWAN scale protocol from Section 6.1 runs in the browser over the **Web Bluetooth** API: the page connects to the scale by name, subscribes to the `ffb2` notify characteristic, and decodes each 8-byte packet with the identical `weight = low + (high | carry) * 256` rule (never masking bit 0 of the high byte, which would drop 256 g). Web Bluetooth is supported on Android Chrome and Edge only; on iOS (all browsers are WebKit) and Firefox it is blocked, so those devices fall back to a manual grams entry. This is a browser policy limit, not a protocol one.

7.12. Limitations of the Web Build

The browser port is honest about its edges. **Web Bluetooth** is Android-only, so iPhone weight is manual. **Latency is CPU-bound**: a flagship finishes both models in about 0.6 s, a budget phone proportionally slower, which is why the measured time is printed rather than hidden. The **first load** downloads about 50 MB and needs $\geq$ 2 GB RAM; very old low-memory phones may struggle, though the files cache afterwards. **WebGPU** is left off by default because its fp16 rounding can flip borderline classes. And **nutrition and Gemini need connectivity** - only the recognition is fully offline. None of these change the recognition result; they bound the surrounding experience, and each is surfaced to the user rather than concealed.

8. Results and Evaluation

This chapter reports what the system actually achieves. It covers each model's accuracy on unseen data, the router's quality, the full routed system against its baseline and ceiling, the edge-build parity, a real-world field test, the alternatives that were measured and rejected, and finally the system's limitations and known bugs. Every figure here is produced by a committed script and comes from data the relevant model never trained on.

8.1. Model Results

The Global model reaches **88.18% top-1** and **96.94% top-5** on its 132-class held-out test set. Critically, its validation accuracy (88.16%) and test accuracy (88.18%) agree to within 0.02 points - the signature of clean, non-leaking splits, and a direct improvement over the earlier model whose 4-point val/test gap revealed train/test leakage. The shipped Israeli specialist (V2, 13 dishes + an open-set **background** class) reaches **88.86%** top-1 (best epoch 112), with top-5 near 98.6%. Its predecessor - a 13-class model with no background class - scored a higher 92.26%, but the background class is what supplies the arbiter's "not-mine" routing signal, so we deliberately traded roughly three points of raw specialist accuracy for a working ensemble (Decision D-08). The remaining errors are sensible visual overlaps (chocolate mousse vs cake, gnocchi vs ravioli) and the smallest ingredient classes, exactly where the least training data exists.

Model	Classes	Top-1	Top-5
Global (general cuisine)	132	88.18%	96.94%
Israeli specialist (shipped, V2)	13 + background	88.86%	98.6%

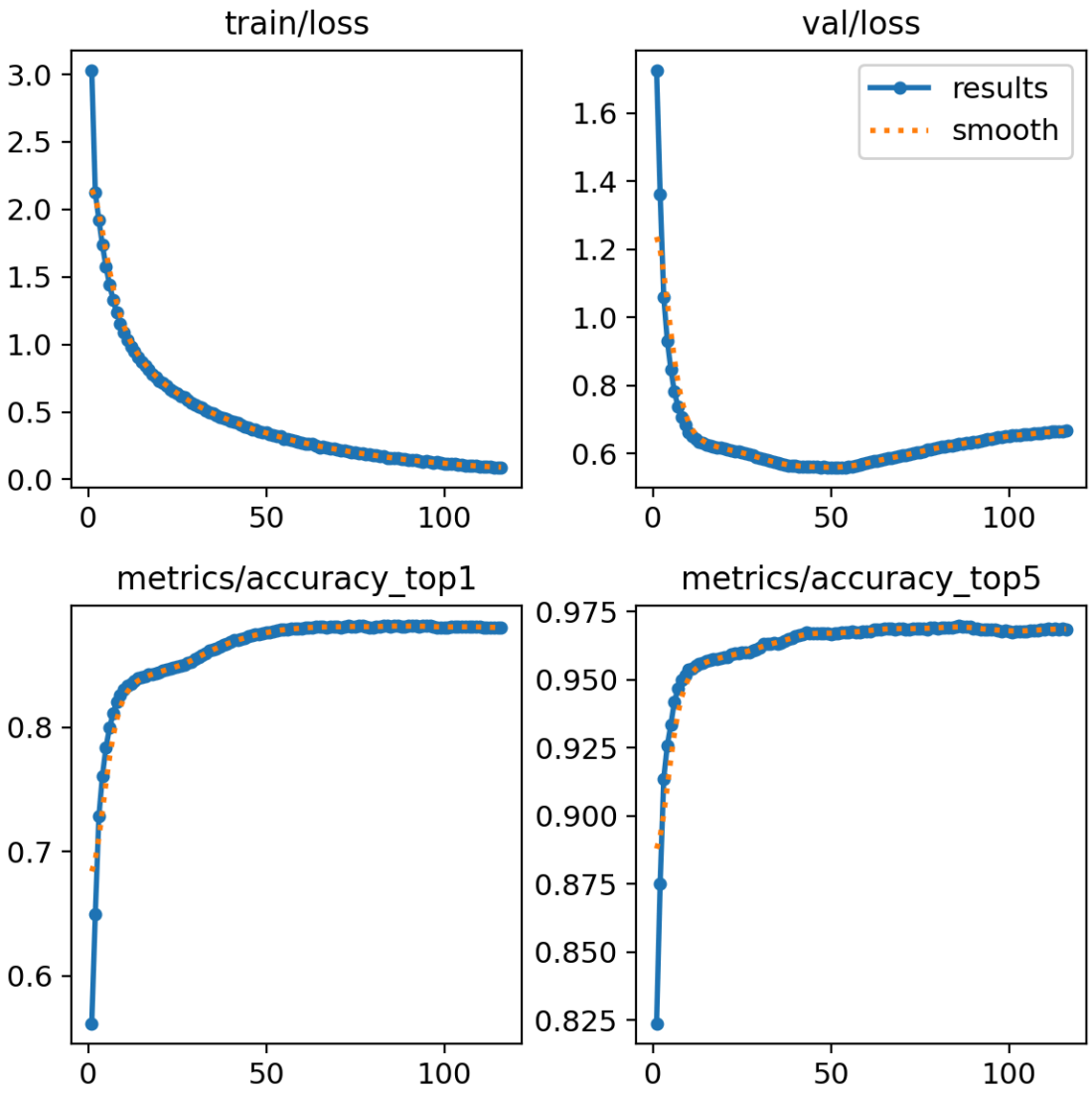


Figure 8: Global model training. Top-1 and top-5 accuracy rise and plateau while training and validation loss fall together; best weights are frozen at the validation peak (epoch 73).

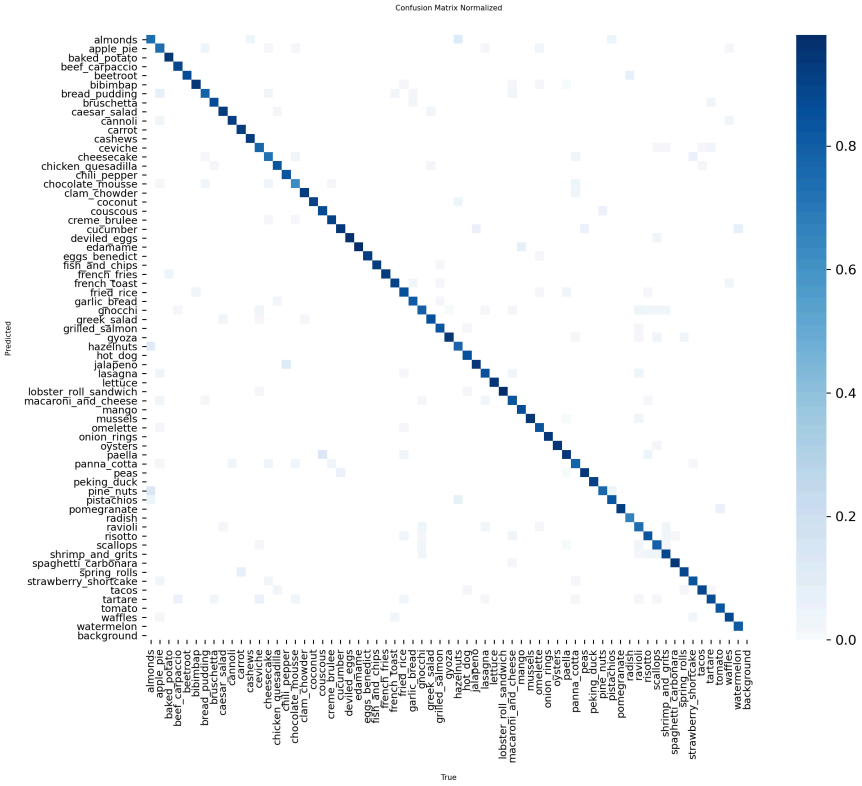


Figure 9: Normalised confusion matrix for the 132-class Global model (test split). The strong diagonal shows correct classification dominates; off-diagonal mass sits on sensible visual overlaps. Full-resolution version is in the repository.

Figure 10: Example validation predictions from the Global model (a single batch).

The Israeli specialist behaves cleanly on its own 14-way problem (13 dishes plus the open-set background class), with a strong diagonal and well-controlled training curves.

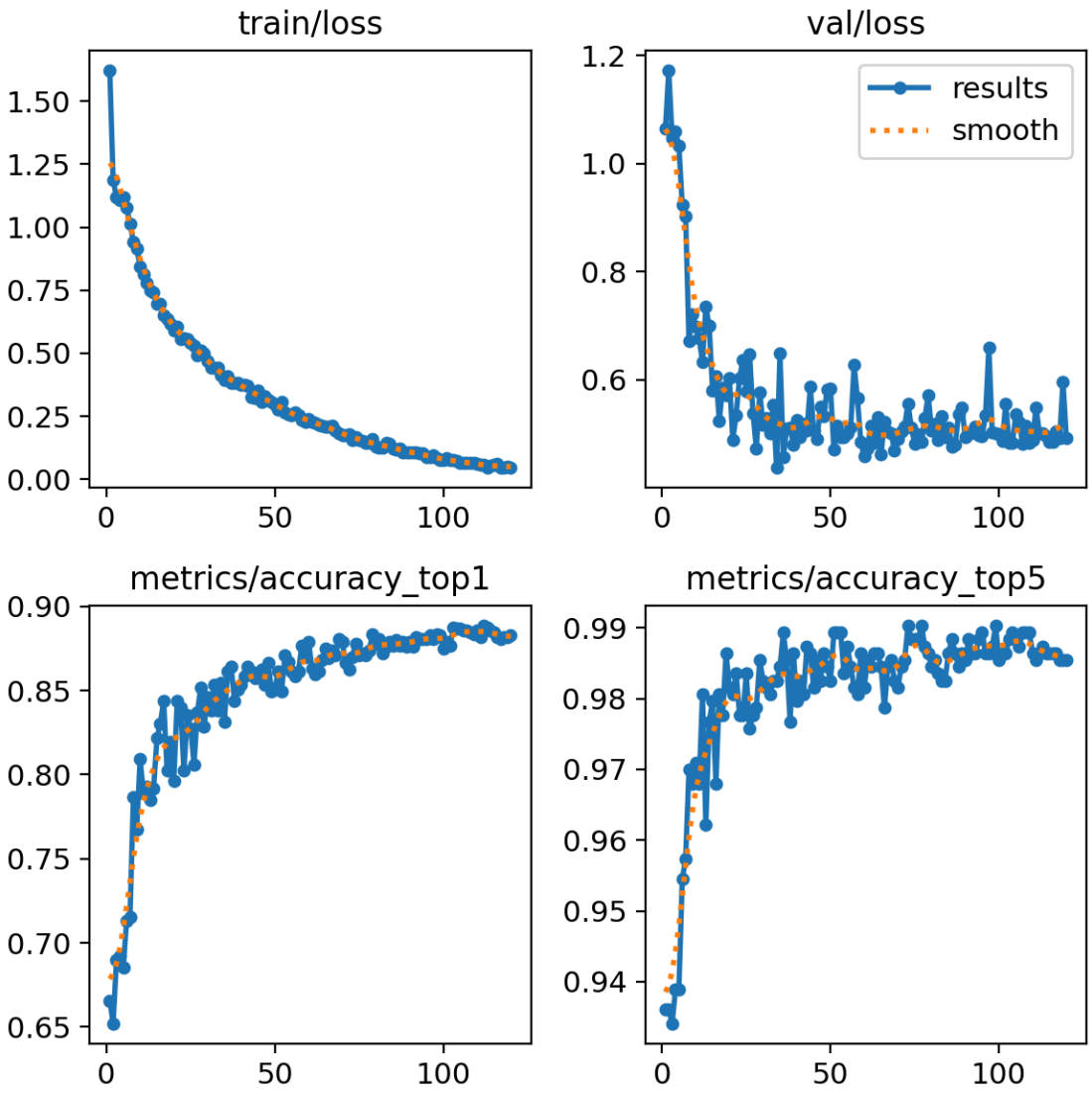


Figure 11: Israeli specialist (V2) training curves.

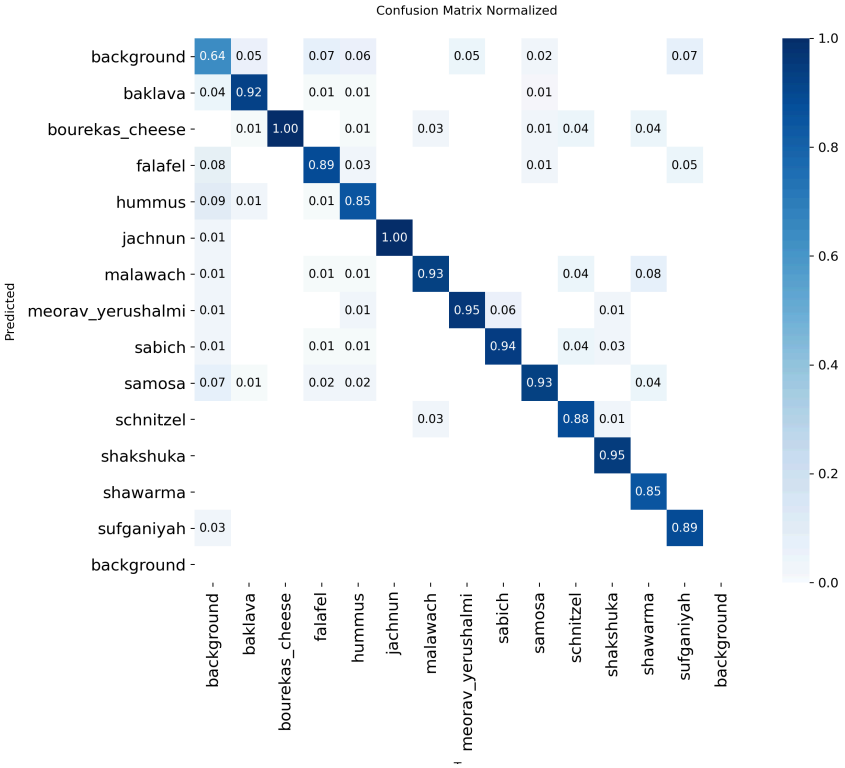


Figure 12: Normalised confusion matrix for the Israeli specialist (V2), including the open-set background class used by the router.

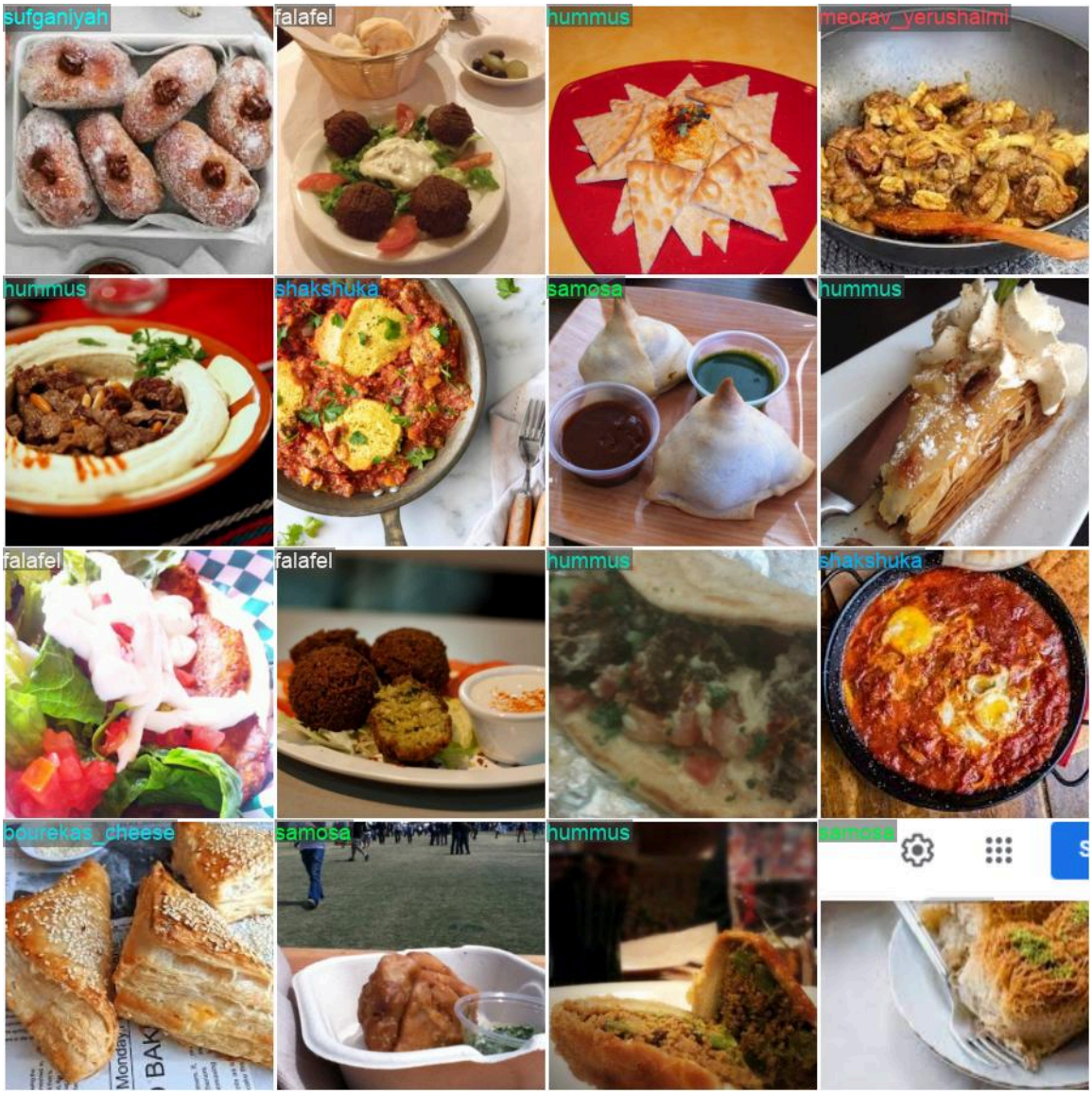


Figure 13: Example validation predictions from the Israeli specialist (a single batch) - the local dishes the Global model was never trained on (hummus, sabich, malawach, shakshuka, bourekas), dishes (hummus, falafel, shakshuka, bourekas, sufganiyah, meorav yerushalmi, samosa), named on held-out photos.

The two mosaics above tell the ensemble story in one glance: the Global model covers 132 world foods, while the Israeli specialist owns the local dishes that motivated the whole two-expert design.

8.2. Router and Full-System Results

The router is a binary domain classifier evaluated on held-out test rows. Adding the open-set **background** signal as a feature (V2) lifted every routing metric over the original design (V1), and the gain came from **information**, not from threshold tuning, which is zero-sum. The most important consequence is the Israeli recall: the fraction of genuine Israeli images correctly sent to the specialist rose from 61% to 79%.

Metric	V1	V2 (shipped)
Routing accuracy	93.4%	96.1%
ROC-AUC	0.933	0.973
Israeli recall	60.8%	79.2%
System top-1 (routed)	83.75%	86.2%

The "money metric" is the routed **system** top-1, compared against two references. The **always-Global baseline** (never route) scores 77.4%; the **oracle** (a perfect router that picks the right expert whenever either model is correct) scores 88.8%. The shipped system reaches **86.2%**, comfortably above the project's 80% target. (Every number in this section is recomputed directly from the committed per-image evaluation table, `datasets/system_evaluation.csv` - 11,352 held-out test rows, one per image.)

System configuration	Top-1
Always-Global baseline (no routing)	77.4%
CalEyeZ routed system (V2)	**86.2%**
Oracle (perfect router) - ceiling	88.8%

The per-domain split is the honest way to read the blended number, because the test set is dominated by global images. On the **global** domain the routed system holds 87.3% against the Global model's own 88.2% - routing costs 0.9 points there, the price of occasionally misrouting a global image to the specialist. On the **Israeli** domain the baseline is **0% by construction** (the Global model does not know those classes at all), and the routed system delivers **77.9%** against a 93.2% domain oracle. That asymmetric trade - give up 0.9 points on one domain to gain 78 points on the other - is the entire value proposition of the ensemble, and it is visible at a glance:

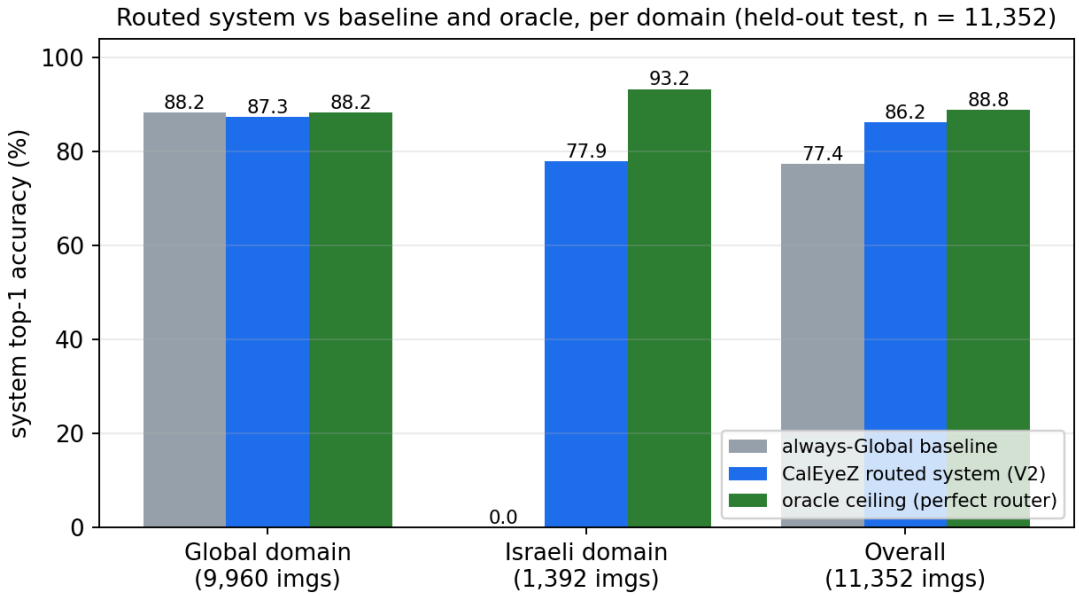


Figure 14: Baseline vs routed system vs oracle, per domain, computed from the committed 11,352-row evaluation table. The Israeli baseline is exactly 0% (disjoint label spaces); routing buys that domain back for a 0.9-point cost on the global domain.

The gap between the baseline and the oracle is the total accuracy the routing problem can recover; CalEyeZ closes most of it. The remaining gap to the oracle is small, and the error decomposition explains why: of the 13.8 points of system error, only **2.7 points** (about one **fifth** of all errors) are **routing** mistakes - the router chose the wrong expert while a correct one existed - and **11.2 points** (about four fifths) are cases where **both models are wrong**, which no router, however perfect, can fix. In other words, the system is limited by the **models**, not the router - to go beyond 88.8% one must improve the classifiers (more or cleaner data, a larger backbone), not tune the arbiter further.

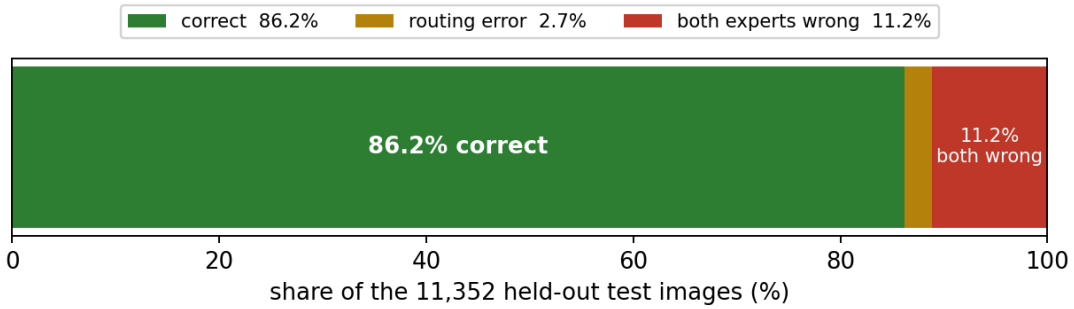


Figure 15: Where the error lives: of 11,352 test images, 86.2% are correct, 2.7% are routing errors (recoverable by a better router), and 11.2% are both-experts-wrong (recoverable only by better models). The router has already closed 77% of the baseline-to-oracle gap.

8.3. Edge Build: Parity and Latency

The ONNX edge build was validated against the PyTorch models so the torch-free deployment can be trusted. Over 200 test images per model the top-1 predictions matched with **0% mismatch**, and the full pipeline (two ONNX models plus the XGBoost router) agreed on routing for every tested image, with a maximum probability difference of about 0.013. On a CPU the build runs an analysis in roughly **0.35 s**, versus 1–3 s for the PyTorch CPU path, confirming the edge build is both faithful and fast enough for interactive use.

8.4. Real-World Field Validation

The strongest test is genuinely unseen data. We ran phone photographs of real plates - outside the dataset, in ordinary lighting - through the deployed pipeline, using the containing folder as ground truth. On an initial set of 40 photos the system scored **30/40 = 75%**, with a 95% Wilson confidence interval of **[59.8%, 85.8%]**; the expanded 47-photo set scored **40/47 = 85.1%** (95% Wilson CI **[72.3%, 92.6%]**), and this is a floor: in the live system the confidence gate routes the low-confidence misses to a fallback instead of reporting them. The per-class breakdown is informative: everyday foods did very well (french fries 3/3, hamburger 3/3, cheese bourekas 9/10), while the misses cluster on under-represented or deliberately degraded inputs.

Class	Score	Typical failure cause
french_fries	3/3	-
hamburger	3/3	-
bourekas_cheese	9/10	1 confused with an Israeli look-alike (malawach)
pizza	5/7	2 deliberately hue/temperature-degraded, dim
canned_tuna	8/11	far / top-down shots lose detail
grapes	2/3	1 low-confidence (macarons)
hummus	0/3	presentation-domain shift: plastic takeaway container vs the restaurant-plated training data

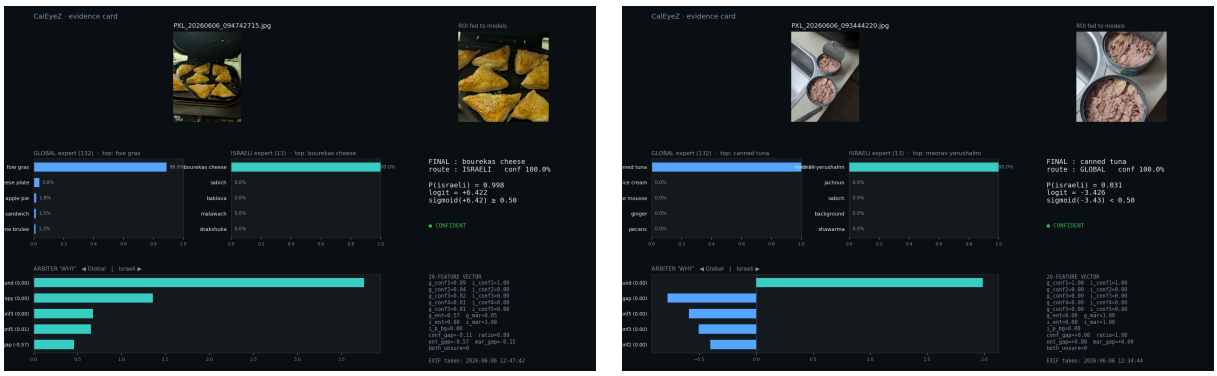


Figure 16: Example field-test evidence cards: a real phone photo with the system's prediction, confidence and routing decision recorded against the folder-level ground truth.

Most misses were **low-confidence** predictions, which the live confidence gate would flag for a fallback rather than report as fact - so the deployed behaviour is better than the raw top-1 count suggests.

The hummus failure (0/3) deserves a precise diagnosis, because it is **not** random weakness. All three field photos showed hummus in a **plastic takeaway container**, while the training data shows hummus **plated in restaurants**. This is textbook **presentation-domain shift** (covariate shift): the classifier learned the dish **in its typical serving context** - the plate, the olive-oil swirl, the garnish - and a new container moves the input off the training distribution, exactly the out-of-distribution failure mode discussed in Section 3.3.11. The field test therefore accidentally ran a controlled OOD-presentation probe, and the model failed the way the theory predicts it should. The diagnosis also sharpens the fix: not merely **more** hummus photos but **more diverse presentations** - containers, home plates, pita-side servings, top-down angles - a data-**diversity** instruction, not a data-**volume** one.

8.5. Weight Channel Calibration

Because calories are computed as per-100 g $\times \frac{\text{weight}}{100}$, weight error passes **linearly** into calorie error: a scale that reads 16% low turns a 300-kcal plate into 252 kcal. The weight channel was therefore validated as a self-contained instrument (Experiment B). Testing the commercial BLE scale against reference masses exposed a fault: the load cell read a **steady $\approx$ 16% low** across the whole range. Crucially the error was **systematic, not random** - repeated placements of the same mass agreed to within a few grams - which means it is a multiplicative **span error**, and a systematic error is correctable in software.

The experiment. Ten reference masses from 21 g to 1062 g (water measured on an accurate kitchen scale) were each read five times on the BLE scale's own LCD, at a fixed centred placement (a spreader plate removes the single-load-cell eccentricity effect). To test that any fit **generalises** rather than curve-fitting water, we then measured **ten completely different everyday objects** - a phone, a battery, a drinking glass, a remote, cardboard, a camera and some toys - as a **held-out validation set** the fit never saw. A perfect scale would place every point on the dashed $y = x$ line; every measured point sits **above** it (the scale reads low), and both sets fall on the **same** straight line through the origin, which is the visual signature of a pure multiplicative span error.

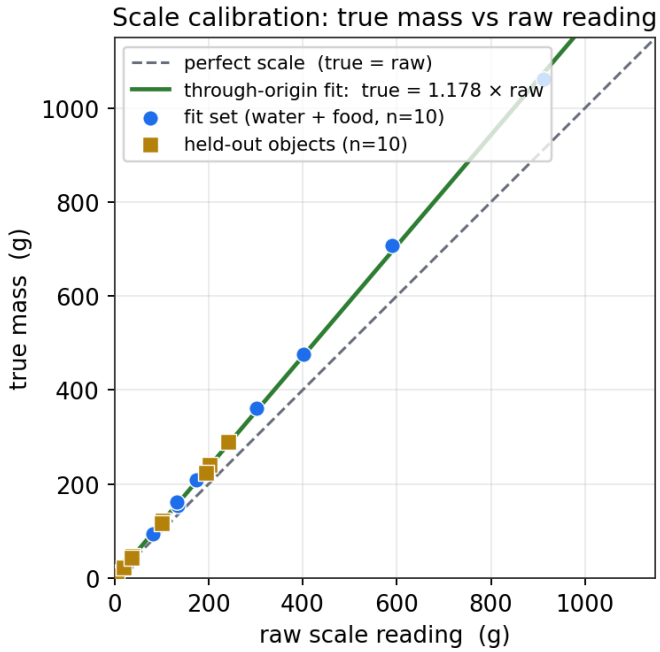


Figure 17: True mass vs raw reading. Blue = fit set (water + food, $n = 10$); amber squares = held-out everyday objects ($n = 10$). All points lie on one through-origin line, well above the dashed perfect-scale diagonal - a constant **proportional** under-read, not a random or additive one.

The regression - exactly what was computed. Each candidate correction maps a raw reading onto true mass and is scored by its **mean absolute error (MAE)** over the $n = 10$ fit masses - the average gap between the corrected reading and the truth:

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|\hat{y}_i - \mathrm{true}_i|$$

Two candidates were fit to the same masses by least squares and compared:

Model	Equation	Fitted numbers	MAE	Max err
No correction (raw)	$\hat{y} = \mathrm{raw}$	none	50.7 g (16.1%)	150 g
Two-parameter line	$\hat{y} = a + b \cdot \mathrm{raw}$	$a = +2.846$ g, $b = 1.1728$	4.9 g	12.7 g
Through-origin (shipped)	$\hat{y} = k \cdot \mathrm{raw}$	$k = 1.17804$	5.1 g	12.5 g

The two fitted models are **tied on error** (4.9 vs 5.1 g), but the two-parameter line carries a spurious **+2.846 g intercept** that over-reads light items: it would turn a true 0 g into 2.85 g and a 53 g portion into 64 g. A load-cell span error is physically **multiplicative**, so the honest model is forced through the origin ($a = 0$), and its single gain k is the least-squares through-origin slope - literally the two sums from the data divided:

$$\text{corrected}_g = k \cdot \text{raw}, \quad k = \frac{\sum_i \text{raw}_i \cdot \text{true}_i}{\sum_i \text{raw}_i^2} = \frac{1,775,157}{1,506,877} = 1.178$$

Applied in software immediately after the five-sample median filter (the reverse-engineered byte protocol is left untouched), this collapses the error from a **16% span** to a **flat few grams** across the entire range. The raw error grows **linearly with load** - the signature of a span error, reaching 150 g at 1 kg - while the corrected error stays near zero and no longer trends with mass:

Figure 18: Absolute weight error vs mass, before and after the 1.178 × correction. The raw error (red) rises linearly to 150 g at 1 kg; after correction (green) it is a flat few grams across the whole 5-1062 g range.

Held-out validation - the constant generalises. The constant was fit on **water and food only**. Applying the **unchanged** $k = 1.178$ to the ten never-seen objects predicts their true mass to a **mean absolute error of just 1.9 g** (mostly within ±1-3%), across metals, glass, plastic and card. Refitting the slope on all 20 objects barely moves it (1.17804 → 1.17825), so the shipped constant is left unchanged - a genuine generalisation test, not a curve fit.

Held-out object	True (g)	Mean raw (g)	$1.178 \times$ raw	Error
tiny screwdriver	3	0.0	0	-3 (below floor)
tiny screwdriver ×2	5	4.2	5	0 (0%)
ball toy	23	20.6	24	$+1$ (+4.3%)
carrot toy	44	37.6	44	0 (0%)
cardboard	47	38.6	45	-2 (-4.3%)
camera	116	100.2	118	$+2$ (+1.7%)
remote	123	103.0	121	-2 (-1.6%)
battery	224	194.2	229	$+5$ (+2.2%)
phone	241	202.4	238	-3 (-1.2%)
drinking glass	289	242.6	286	-3 (-1.0%)

Note: Finding - a hard detection floor near 4 g. The 3 g screwdriver read **0 g on every one of five attempts**: the scale cannot sense a load that small, and **no multiplicative correction can recover a zero** ($1.178 \times 0 = 0$). Doubling it to 5 g was detected correctly. The honest specification is therefore a **practical lower limit of ≈ 5 g**; below it the weight - and any calorie figure derived from it - is unreliable. This matters only for near-weightless items (a pinch of spice, a single sweet) and is surfaced as a limitation rather than hidden.

Final calibration and error budget. The shipped formula $\text{corrected}_g = 1.178 \times \text{raw}$ is **identical** in the desktop demo, the ONNX edge executable, and the browser app. Validated on all 20 objects it gives a mean absolute error of **3.6 g** (down from 34.0 g / 16-20% raw); on the **held-out** objects alone the error is **1.9 g**. Max error is $\approx \pm 13$ g over the full 5-1062 g range - best mid-range ($\approx$ 1-3%), with only the smallest masses showing a larger **percentage** (a few grams on a 20 g item). The residual is now dominated by the scale's own **repeatability** (mean spread $\approx$ 10 g, up to 31 g at 1 kg), not by the fit: the correction has reached the **hardware noise floor**, and the median filter smooths what remains. This is why the app deliberately displays a **different** number from the scale's LCD - the LCD shows the uncorrected raw value, the app shows calibrated true mass. Source data: `scripts/eval/weights_cal.csv`, fit by `scripts/eval/weight_calibration.py`. The correction turns an off-the-shelf faulty scale into a specified instrument: $\approx \pm$1-3% **above** $\approx$ 100 g, and gram-level **absolute** error (a few grams, hence a larger percentage) below it, down to the 5 g floor.

8.6. End-to-End Calorie Validation (Experiment A)

The final claim of the system is a calorie figure, so the last experiment validates exactly that: real meals, weighed and identified live by the deployed app, against a **ground truth** built from verified sources (nutrition labels, USDA entries for verified foods, and recipe decomposition for prepared dishes). Following Decision D-24, each trial's error is **decomposed by channel** - identity, weight, and database - because a single end-to-end percentage hides where the error actually lives. The experiment is ongoing; the first six logged meals already show the structure clearly:

Meal	Identified	Weight err	Database err	Total kcal err
grapes	✓	2.1%	0.02%	2.1%
bell pepper	✓	1.8%	26.8%	25.4%
cucumber	✓	1.2%	34.2%	35.0%
canned tuna (label truth)	✓	3.1%	11.5%	8.0%
french fries	✓	3.8%	126.6%	135.2%
hummus	✓	3.2%	40.7%	38.8%

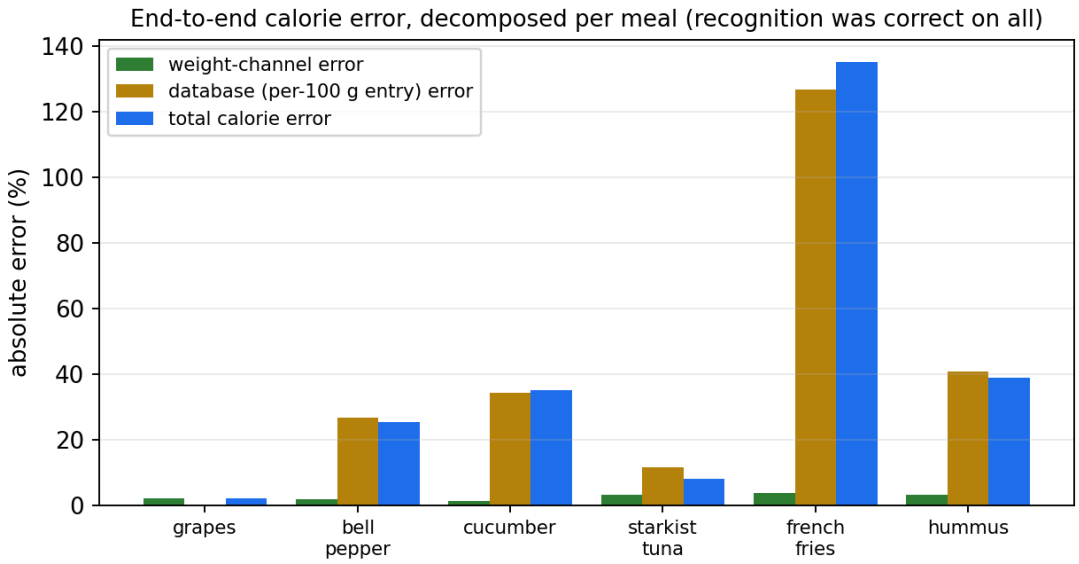


Figure 19: Per-meal error decomposition (from `scripts/eval/calorie_validation_results.csv`). The green weight-channel bars are uniformly small (1-4%); the total calorie error tracks the amber **database** bars almost exactly - the residual error is the per-100 g entry choice, not the instrument.

Three findings, each visible in the chart. **First, recognition is not the problem**: all six meals were identified correctly, consistent with the field test. **Second, the weight channel is solved**: after the Section 8.5 calibration its error is 1.2-3.8% (mean 2.5%), and it never dominates a single trial - the direct payoff of the BLE pivot and the span correction. **Third, the residual error is the database**: the total calorie error tracks the database-entry error almost one-for-one. The extreme case is instructive - the french-fries trial hit 135% total error because the database's generic fries entry carries roughly 2.3 × the energy per 100 g of the verified truth for the actual meal; the **same** trial's weight error was 3.8%. Percentage errors are also inflated on near-zero-calorie foods by construction (a 4.3 kcal absolute miss on a cucumber reads as 35%), which is why absolute kcal is reported alongside.

The engineering conclusion mirrors the classifier ceiling analysis: the channels this project **built** - recognition and weight - are validated to field accuracy, and the remaining error is concentrated in the third-party nutrition **data**, the irreducible term that motivated the three-tier lookup and the evidence-based cooking multipliers (Sections 7.9-7.10). A diagnosis, not just a score.

8.7. Benchmarking Against the Market (Experiment D)

Experiment A proved the calorie figure is **internally** honest; the obvious next question is **external**: how does CalEyeZ compare to the tools a user would otherwise reach for? We ran a head-to-head pilot against the three products people actually use - MyFitnessPal (manual database logging), **Cal.ai** and **FoodVisor** (cloud AI photo estimators) - on the **same physical meals**, with the product label as ground truth wherever one exists. Every number below is a logged trial in `scripts/eval/calorie_comparison.xlsx`; this is a deliberately honest **pilot** (small n), and each figure's source is stated.

8.7.1. The headline: accuracy on shared meals

Across the eight meals all three tools were run on, CalEyeZ had the **lowest** mean absolute calorie error - and it earned that while conceding every advantage to the competition.

Tool	Mean kcal err	How it gets its portion
CalEyeZ	**37%**	measures grams on a BLE scale
MyFitnessPal	49%	user weighs and types the true grams (generous manual case)
Cal.ai	64%	cloud model guesses a typical serving from a photo
FoodVisor	–	photo estimate; paywalled after one free image

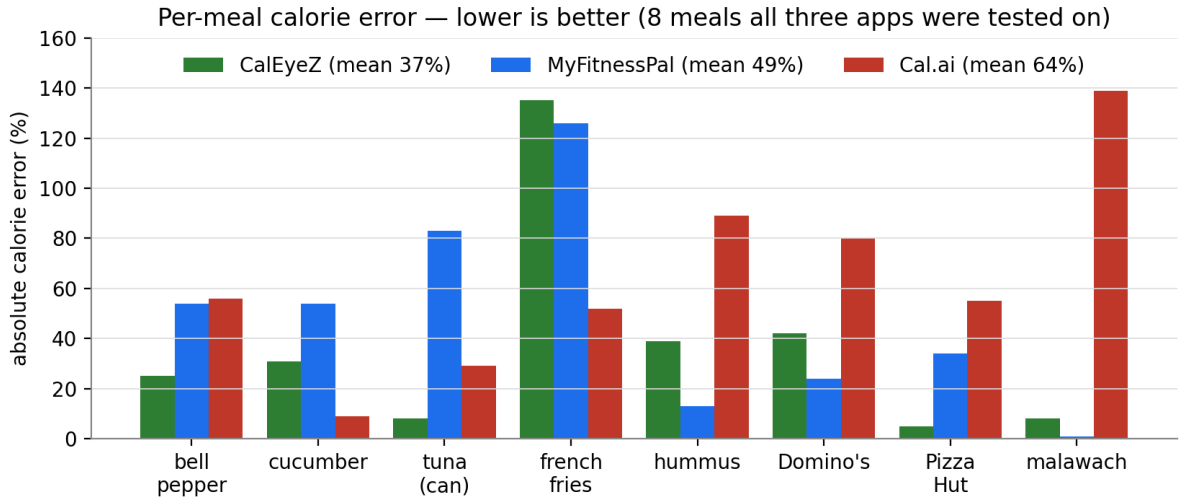


Figure 20: Per-meal absolute calorie error for the three apps on the eight shared meals (`scripts/eval/calorie_comparison.xlsx`). CalEyeZ wins or ties most rows; the one row where it looks worst - french fries - is a **database** miss (generic deep-fried entry vs a home-fried meal), not a measurement miss, mirroring Experiment A.

The result is notable because **MyFitnessPal was handed the true weight and a hand-picked database entry**, and **Cal.ai is a giant cloud vision-language model** - yet the on-device scale-based system was more accurate than both. The competitors' occasional wins (hummus, malawach for MyFitnessPal) come from a **mature crowd-sourced food database**, an entry-quality advantage that is fixable on our side and orthogonal to **measurement**; they never come from a better estimate of **how much food is on the plate**.

8.7.2. The decisive test: does the number follow the portion?

The whole problem reduces to **portion**. A photo app emits a **typical serving**; a manual app offers **preset units** ("1 slice", "1 can"). Both are correct only when the food happens to match their assumption. CalEyeZ reads the actual grams, so its calorie figure tracks the real portion. We isolated this with a single food at two portions - a full can of tuna (162 g) and a small 14 g ball:

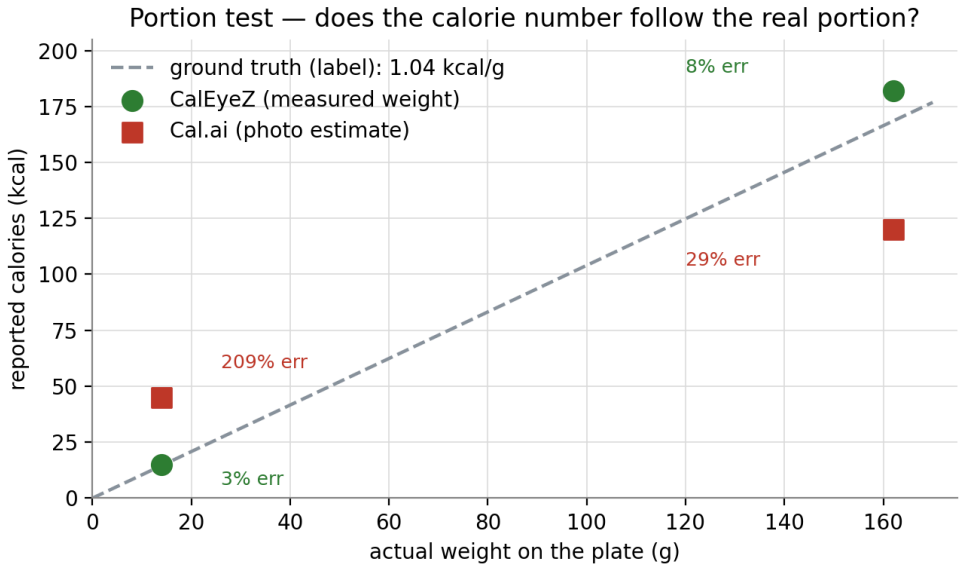


Figure 21: Reported calories vs the **actual** weight on the plate, canned tuna. CalEyeZ (green) sits on the ground-truth diagonal at both portions (3% and 8% error); Cal.ai (red) is nearly flat - it read the 14 g ball at 45 kcal (**209%** error), never leaving its "small serving of tuna" prior. It is not measuring; it is guessing a standard serving.

This generalises past tuna. A photo shows a food's **top surface**, not its mass: a thin minute-steak and a thick ribeye can cover the same plate area yet differ two-to-threefold in grams - invisible from above. That is the identical **self-occlusion / unknown-density** wall that made us reject stereo and LiDAR depth (Section 8.7). A state-of-the-art cloud model fails here for the **same physical reason** as depth hardware: pixels carry appearance, not weight. A scale reads the weight directly. The same failure surfaced as Cal.ai returning an **identical 425 kcal for both a Domino's and a Pizza Hut slice** - excellent recognition, but a fixed quantity, because it has no way to weigh the food.

8.7.3. The axis no competitor contests: on-device, offline, instant

Beyond accuracy, CalEyeZ occupies a corner of the design space the cloud apps cannot reach. Its recognition runs **on the device**: verified by keeping the app identifying food with networking disabled (airplane mode), whereas Cal.ai and FoodVisor stop working offline - empirical proof their inference is server-side, and the direct cause of their **5-140 s** latency (the network round-trip) against CalEyeZ's 0.5 s. The pipeline is also **free** (no account, no subscription; FoodVisor paywalls after one photo) and **private** (the image never leaves the device).

Note: Honest limits of this pilot. Small sample (10 meals; Cal.ai on 8, FoodVisor on 1, which is why FoodVisor carries no mean). Ground truth is a real **label** for tuna, fries, hummus, both pizzas and malawach, but only a **reference database** for grapes/cucumber/pepper. Percentage error is inflated on near-zero-calorie foods (a few-kcal miss on a cucumber reads as tens of percent). We never tune the database to the test meals - that would be leakage - so the residual database error is reported, not hidden. The takeaway is not "CalEyeZ is universally best" but the structural one: **recognition can be solved many ways; calories can only be solved by measuring the weight**, and on the same meals CalEyeZ is the only tool whose number follows the actual portion.

8.8. Viewpoint Repeatability (Experiment E)

Experiment D measured **accuracy** - closeness to the truth. But a measuring instrument must first pass a more basic test: **repeatability**. Given the identical input twice, does it return the same answer? A tool that does not is not measuring; it is guessing. We isolated this cleanly. A single plate of **StarKist canned tuna in water** (104 kcal/100 g) was photographed seven times, changing **only the camera angle and the layout on the plate** - the food and its weight never moved. Both apps were run on each arrangement.

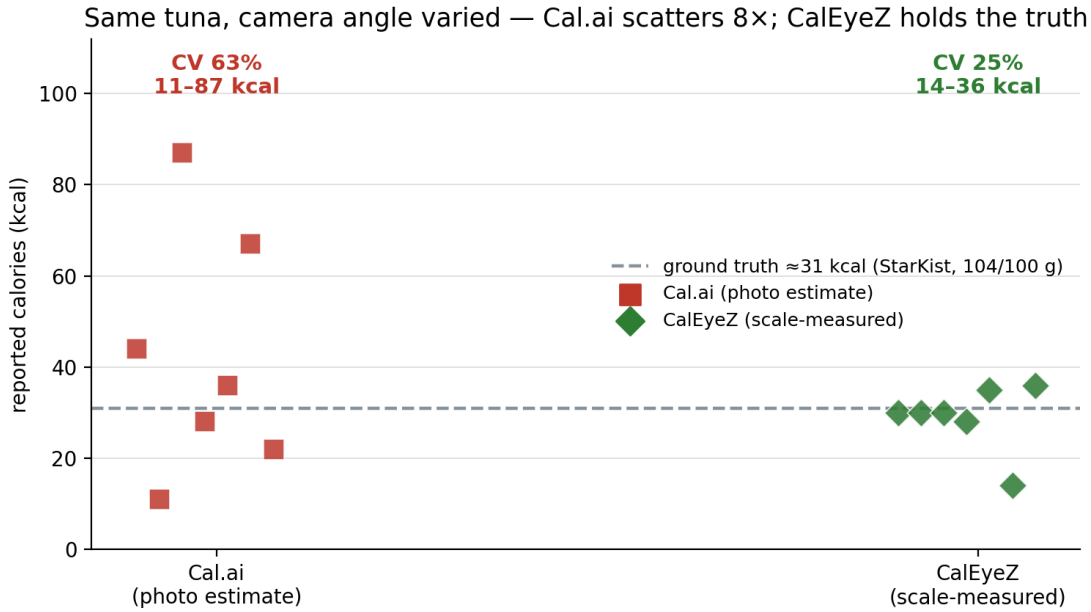


Figure 22: Seven photos of one tuna plate (truth ≈ 31 kcal). Cal.ai's calorie output swings across the full range while CalEyeZ stays on the truth line - because CalEyeZ reads the weight from the scale, which does not change with viewpoint, whereas Cal.ai infers the portion from pixels, which do. Data: `scripts/eval/repeatability.csv`.

Cal.ai is not self-consistent. On the same tuna it returned **11, 22, 28, 36, 44, 67 and 87 kcal** - a coefficient of variation of **63%** and an eight-fold spread - and on one arrangement it renamed the dish "Tuna and Tofu Preparation." The calorie number is a typical-serving prior applied to whatever the camera happened to see; change the view and the prior changes. This is the same self-occlusion and unknown-density wall analysed in Experiment D and Section 8.7, now expressed as pure **noise** rather than bias.

CalEyeZ holds the truth, and reports its failure modes honestly. Six of the seven readings fell in 28-36 kcal, on the ground-truth line; its coefficient of variation is **10%** (excluding one outlier, **25%** including it). The lone 14 kcal outlier was a **nutrition-database** error from the cloud fallback returning a low per-100 g value, **not** a weight error - the measured grams were correct on every shot, which is exactly the channel decomposition of Experiment A. Two honest observations on the recognition channel, both visible in the logged captures. First, the on-device classifier was frequently **unsure** on this food: shredded tuna is visually close to the Israeli dish **meorav yerushalmi**, so the Israeli expert over-fired and the arbiter's confidence gate correctly escalated to the Gemini-vision fallback (Section 7.9) rather than emit a low-confidence guess - the safety net behaving as designed, not a defect. Second, this was not universal: on one arrangement the Global expert identified **canned tuna** outright at **99%** confidence with no fallback needed. The system is therefore honest about its uncertainty and still lands the calorie figure, because the weight anchor does not depend on the recognition being easy.

This is the sharpest single result in the comparison. It does not claim CalEyeZ is uniformly more accurate; it shows that the competitor **cannot reproduce its own answer** for one unchanging meal, while CalEyeZ can - and that the difference is structural, because one instrument **measures** the portion and the other **estimates** it.

8.9. Evaluated Alternatives - Tried and Rejected

Before choosing a BLE scale for weight, we surveyed the full weight-sensing design space. Two camera-based routes we **built and measured** in the lab; a third branch - active and stereo depth hardware - we rejected at the design stage for the reasons below. All were rejected for concrete engineering reasons. Recording this is the honest engineering answer to "why a scale and not just the camera?" The common thread: a camera is excellent at telling you **what** a food is, but a poor instrument for **how much** it weighs. Every vision route to mass runs through a lossy chain, and each link adds error - and every volume route, however the volume is obtained, ends at the same **unknown-density** wall.

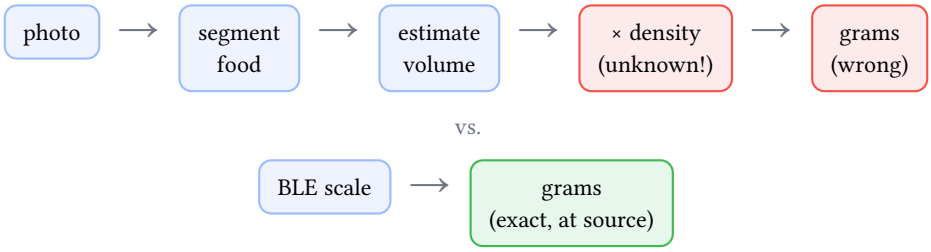


Figure 23: The rejected vision chain compounds error and ends at an unknown density; the chosen BLE path reads the mass the hardware already computes.

Approach A0 - active and stereo depth sensing (rejected at design time, before building). Before the monocular experiments, the obvious way to recover volume is to **measure** depth with dedicated hardware rather than infer it: a **stereo camera pair** (triangulating disparity across a known baseline) or an **active depth sensor** (LiDAR / time-of-flight, timing an emitted IR pulse). Prior food-diary systems have taken exactly this RGB-D route[14], so the option was taken seriously. After a literature and design-space review, however, we rejected both **without prototyping them** - the analysis below shows the effort would not have changed the outcome - for four compounding reasons.

[14] A. Meyers et al., "Im2Calories: Towards an Automated Mobile Vision Food Diary," *ICCV*, 2015, pair an RGB image with a learned/measured depth map to estimate portion volume - and still report portion size as the dominant error source, which is the same wall CalEyeZ hits.

First and decisive: they do not escape the **density problem**. Even a perfect, watertight volume still needs a per-food **density** to become grams, and that is unknown at inference (Approach A) - so better depth hardware buys a more accurate **volume** and leaves the **mass** just as wrong. The entire branch is therefore dominated by direct gravimetry regardless of sensor quality. Second, **self-occlusion**: any single-viewpoint depth sensor sees only the food's **top surface** and returns a height field, not a closed volume; the hidden underside and the contact area with the plate must be **assumed**, reintroducing exactly the modelling error the sensor was meant to remove. Third, **material failure modes**: stereo disparity collapses on the low-texture, glossy and specular surfaces typical of food (a smooth hummus, a shiny apple) leaving holes precisely over the object, while IR time-of-flight is confounded by specular, dark, wet or translucent foods and by multipath on concave plates, and its point cloud is spatially sparse relative to the fine detail volume needs. Fourth, **platform and cost**: a stereo rig adds a second calibrated camera and a rigid baseline, and LiDAR/ToF exists on only a minority of high-end phones - both of which violate the zero-install, runs-on-any-handed-over-device constraint that drove the browser design (Decision D-20). For a consumer nutrition tool the added bill-of-materials, calibration and power simply buy a more expensive path to the same density dead-end.

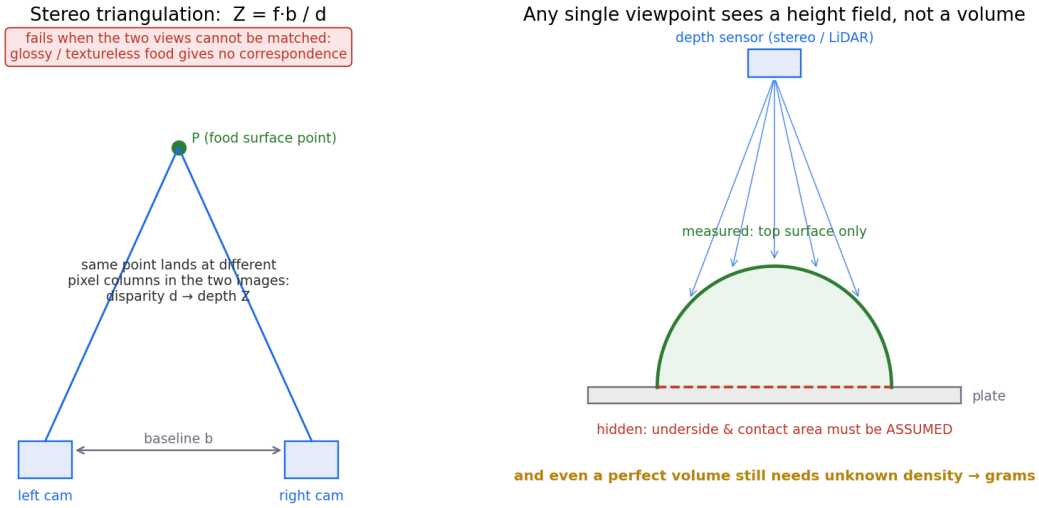


Figure 24: Why depth hardware was rejected without building it. **Left:** stereo recovers depth by triangulating the disparity d of the same point across a baseline b ($Z = fb/d$) - but the match fails exactly on glossy, textureless food surfaces. **Right:** any single-viewpoint sensor (stereo or LiDAR) returns a **height field** of the visible top surface; the underside and contact area must be assumed, and even a perfect volume still collides with the unknown-density wall.

Approach A - volume to mass from a photo. Using a 1-shekel coin (18 mm) as a fiducial to convert pixels to centimetres, we recovered food **volume** by two independent methods and validated them against water displacement (Archimedes, the gold standard). Monocular depth (MiDaS) estimated an apple at **139.3 cm³** against a true 150 cm³ - within about 7%. Shadow geometry (height = shadow length at a 45° light) estimated a can at **519.67 cm³**. The methods were **accurate**; that was never the problem. The fatal flaw is the step **after** volume: converting cm³ to grams needs **density**, which varies enormously between foods (a cup of salad vs a cup of peanut butter) and is **unknown at inference**. So even a perfect volume yields a wrong weight. The methods also required a coin in frame, a single clean object, a plain background, and (for shadows) a hard directional light - none of which survive a real plate.

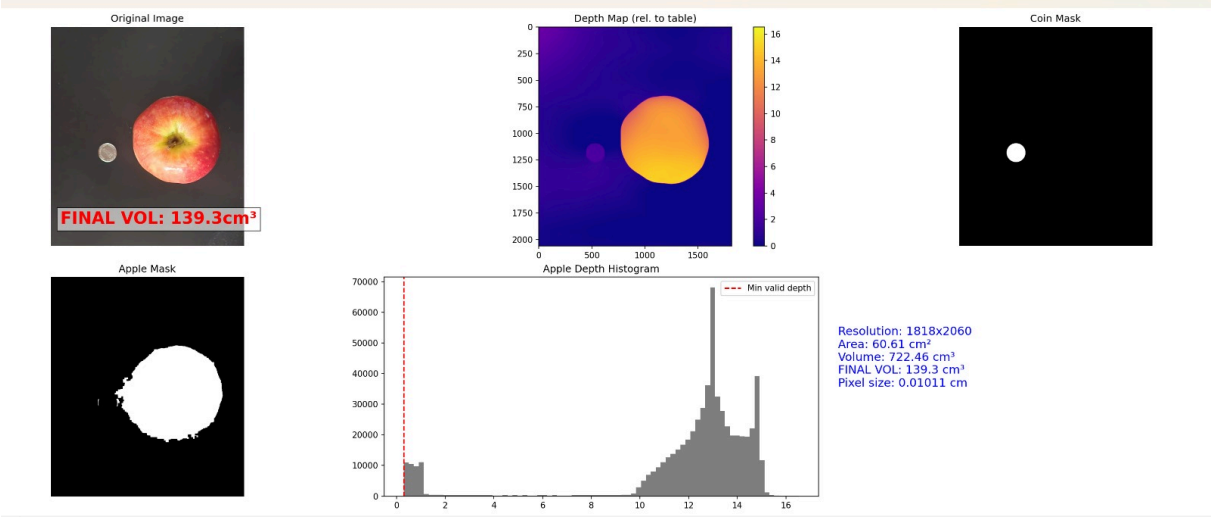


Figure 25: Volume from monocular depth (MiDaS). The coin fiducial sets pixel size; the depth map over the apple mask integrates to a volume of 139.3 cm³, within 7% of the true volume - yet still useless without a density value to convert it to grams.

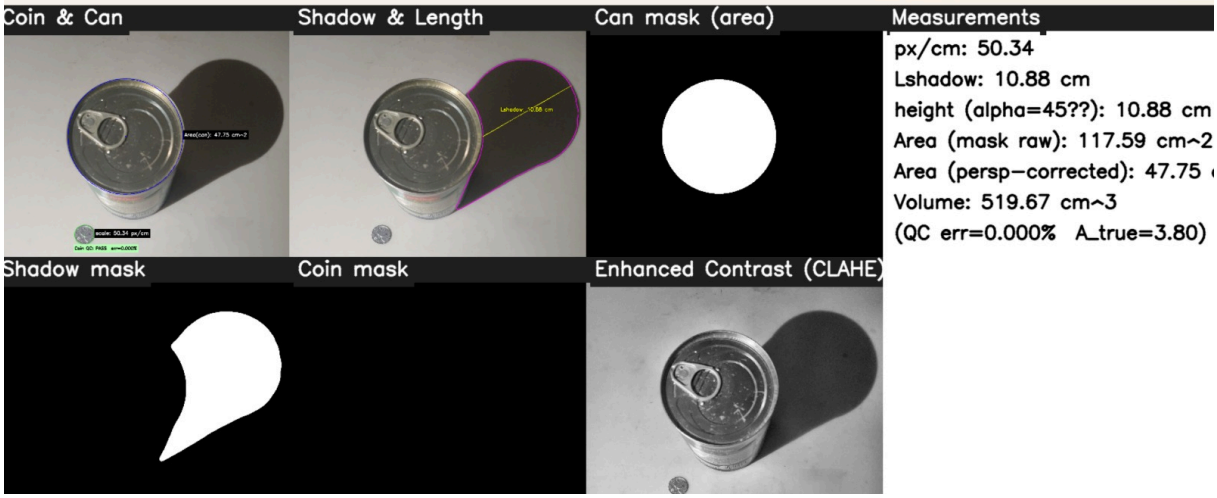


Figure 26: Volume from shadow geometry. With a 45° light an object's height equals its shadow length; the perspective-corrected top area times that height gives 519.67 cm³, again without any depth network - and again stuck at the density step.

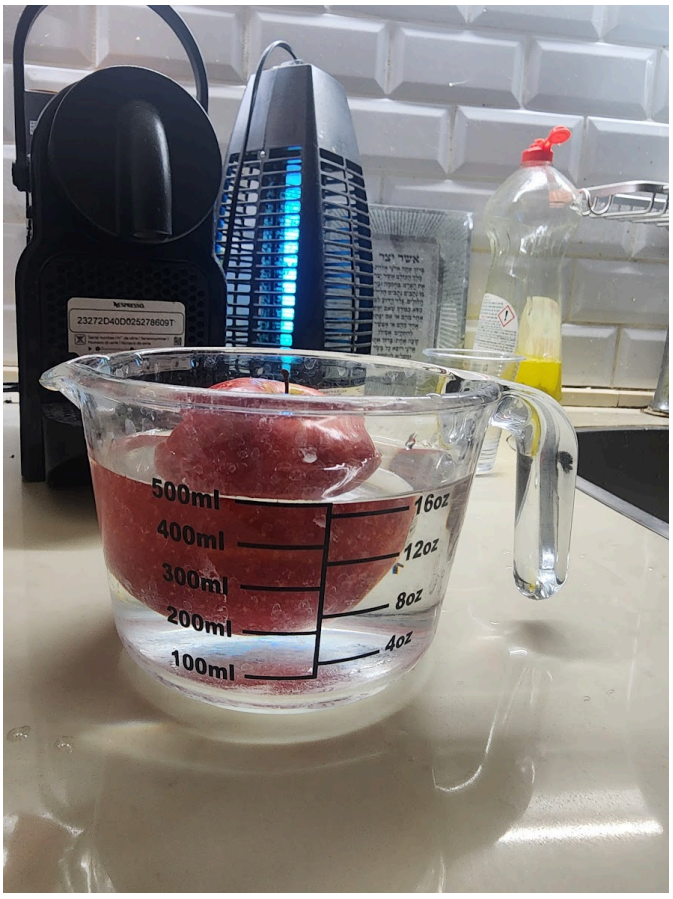


Figure 27: Validation by water displacement (Archimedes), the gold standard. The methods measured volume accurately; the failure is everything after volume, where density is unknown.

Approach B - OCR of the scale's display. Keeping a real scale but reading its 7-segment LCD with OCR (`pytesseract`) avoided pairing, but seven-segment OCR is brittle: general OCR is trained on normal fonts, the LCD glares under room light, a slight angle merges or splits segments, and the value flickers as it settles. We were stacking a fragile vision problem on top of a device that already knew the answer digitally.

The decision. Every rejected route makes the same mistake - inferring a physical quantity the hardware already measures exactly (and, for the volume routes, colliding with unknown density whether the depth is guessed by a network or measured by a sensor). We reverse-engineered the scale's BLE protocol and read the mass directly: no density assumption, no fiducial, no glare dependence, gram-accurate every time.

Approach	Lab result	Why rejected
Stereo depth volume	not built (design-time)	top-surface only (self-occlusion); disparity fails on glossy/low-texture food; still needs unknown density; adds a second calibrated camera
LiDAR / ToF depth volume	not built (design-time)	sparse, IR-confounded on wet/dark/specular food; top-surface only; still needs unknown density; on few phones (breaks D-20)
MiDaS depth volume	apple 139.3 cm^3 (7% vs Archimedes)	volume needs unknown density to become grams; controlled conditions only
Shadow-geometry volume	can 519.67 cm^3	same density problem; needs a coin and a hard shadow
7-segment OCR of scale	worked on clean frames	glare, angle, flicker; segmented-font OCR unreliable
BLE gravimetric (chosen)	**exact grams at source**	**robust; no inference, no assumptions**

Note: The principle, in one line. Use the camera for identity, never for mass. When a physical quantity is already measured by the hardware, read it directly rather than inferring it through a lossy proxy. That single decision is what made the weight side of the system reliable.

8.10. Validation of Functional Specifications

The implemented system meets its headline specifications. The $\geq 80\%$ system-accuracy requirement is met (86.2%). The dual-model + router architecture is validated: it adds local cuisine without degrading the Global model, which is the concrete defeat of catastrophic forgetting. The latency requirement is met on both GPU and the CPU edge build. The one **specification change** from the FRS is deliberate and documented: weight acquisition moved from OCR to BLE after OCR failed its robustness requirement - a change that **improved** the system rather than compromising it.

8.11. Performance Limitations

1. **Closed vocabulary.** Only trained classes are recognised; an unseen food is mapped to the nearest visual match. The live confidence gate mitigates this but cannot eliminate it.
2. **Model ceiling 88%.** About two-thirds of system errors are both-models-wrong, so accuracy is bounded by the classifiers, not the router.
3. **Under-represented classes.** Visually featureless or low-data classes (e.g. hummus) are the weakest, as the field test showed.
4. **Single-item assumption.** The system analyses one dominant food per image; mixed plates are out of scope.
5. **Sensing conditions.** The camera needs adequate lighting, and the weight depends on a stable BLE link to the specific scale; extreme glare or a dropped link degrades the result.

6. **Connectivity for nutrition.** Full macro lookup uses the USDA API; offline operation falls back to a smaller local database.

8.12. Known Bugs

The most significant bug found and **fixed** was the BLE carry-byte decode (masking the high byte's low bit silently dropped 256 g on odd multiples of 256), resolved by correcting the decode and adding a median filter (Section 6.1). A related transient - a momentary overshoot latching a permanent +256 g offset - was solved by median-filtering the raw decodes instead of latching a "sticky high" byte. **Open / accepted limitations:** hummus misclassification is unsolved and attributed to insufficient training data (the fix is more data, not code); and seven-segment OCR was abandoned rather than fixed, because the BLE path made it unnecessary.

8.13. Conclusion

CalEyeZ meets its objectives. It classifies a large, mixed food vocabulary at 86.2% system top-1 - above target - using a router-managed ensemble that demonstrably avoids catastrophic forgetting, and it measures weight deterministically over BLE after rigorously rejecting the vision-based alternatives. The evaluation is honest and reproducible: leakage-free splits, a held-out router test, an edge build verified to parity, and a real-world field test reported with confidence intervals. The clearest path to a better system is now better **models**, not a better router - which is itself a useful, data-backed conclusion.

9. Conclusion

9.1. Interpretation of Results

The central result is that a **team of specialised models managed by a learned router** beats a single general model on a real, mixed food vocabulary. The routed system reaches 86.2% top-1, well clear of the 80% target and 8.8 points above the always-Global baseline (77.4%), while the Global model itself holds 88.18% on clean, leakage-free splits. Just as important is **what the numbers say about where the limit is**: about four fifths of the remaining system errors are cases where both models are wrong, so the system is now bounded by the classifiers rather than the router. That single finding reframes any future effort. Tuning the arbiter is close to exhausted; the gains live in the models and the data.

The weight side tells a parallel story. By measuring mass directly over BLE instead of inferring it from pixels, the system avoids the density problem that defeated the volume methods and the fragility that defeated OCR. The lesson generalises: when the hardware already computes a physical quantity, read it, do not re-derive it.

9.2. Challenges and Lessons Learned

The hardest problems were not the ones we expected. Three stand out.

Catastrophic forgetting forced the whole architecture. Naively extending the Global model to Israeli dishes destroyed its accuracy on the original classes, which is what pushed us toward the two-experts-plus-router design rather than one ever-growing network.

Data leakage taught us to distrust a good test score. The earlier model looked strong until we noticed its validation and test accuracies disagreed by about four points. Exact and perceptual de-duplication followed by a fresh split closed that gap (88.16% vs 88.18%), and only then was the accuracy trustworthy.

The weight subsystem was a sequence of honest dead ends. Volume-from-vision was accurate at estimating volume (within 7% of Archimedes) yet useless without density; OCR of the scale's display was defeated by glare, angle and flicker. Reverse-engineering the BLE protocol was the unglamorous but correct answer, and even that hid a subtle carry-byte bug that silently dropped 256 g until a range-sweep test exposed it.

The overarching lesson is that **measurement honesty matters as much as model design**: leakage-free splits, a held-out router test, edge-parity verification, and field results with confidence intervals are what turn a demo into evidence.

9.3. The Documented Hurdles: a Register of Failures and Recoveries

A project's real engineering story is what went **wrong** and what was done about it. Every entry below is a genuine failure we hit, kept in the project's documentation rather than erased; each was diagnosed to a root cause and either fixed, redesigned around, or - where the honest answer was "abandon it" - replaced. Nothing here is hypothetical, and several of the failures directly produced the architecture the book describes.

#	What went wrong	Diagnosis	Resolution
1	OCR of the scale's 7-segment LCD unreliable under classroom light	glare, viewing angle and settle-flicker defeat segmented-font OCR	**pivoted** the whole weight channel to a reverse-engineered BLE read-at-source
2	Vision weight estimation: volume accurate, mass wrong	volume⟶mass needs **density**, unknown at inference (validated vs Archimedes)	rejected the entire vision-mass branch; BLE gravimetry chosen
3	BLE weights above 255 g read wrong (272 g read as 16 g)	decode masked bit 0 of the high byte, silently dropping 256 g carries	range-sweep test exposed it; decode fixed and re-verified over the working range
4	A transient overshoot latched a permanent +256 g offset	a "sticky high byte" latch turned one bad frame into a lasting error	replaced with a 5-sample **median filter** (robust to spikes by construction)
5	The scale itself read a steady 16% low	systematic multiplicative **span error** in the load cell	calibration experiment (20 objects, held-out validation) $\longrightarrow k = 1.178$ correction; 5 g dead-zone floor documented as a limitation
6	Earlier Global model's test score too good to be true	4-point val/test gap = train/test **leakage** via near-duplicate images	SHA-1 + perceptual dHash de-duplication, fresh 70/20/10 split; val/test now agree to 0.02 pts
7	Single model extended to Israeli dishes lost its original classes	**catastrophic forgetting** - new-class gradients overwrite old weights	two frozen experts + a learned router; forgetting eliminated by construction
8	Closed-set Israeli model "shouted hummus" at everything	softmax overconfidence on out-of-distribution inputs (a known ReLU-network failure)	V2 retrain with an open-set **background** class; its $P(\mathrm{bg})$ became the router's top feature, AUC 0.933⟶0.973

#	What went wrong	Diagnosis	Resolution
9	Threshold tuning failed to improve the system	swept 0.30-0.55: accuracy flat - threshold moves only **redistribute** error	gain had to come from **information**, not tuning - which the background feature then delivered
10	Four fine-tunes on real-world captures all degraded the model	each learned the new samples but **regressed the clean test** (down to 23-66%)	automatic regression gate refused all four; production weights stayed intact; full retrain adopted as the only safe path
11	Test-time augmentation (TTA) made the system **worse**	averaging raw+CLAHE softmax shifted confidences away from what the arbiter was trained on - it broke **routing**, not classification	reverted; lesson recorded: any change to the experts' outputs invalidates the arbiter's training distribution
12	Demo-time segmentation (GrabCut, border cropping) hurt accuracy	the classifier is already background-robust; cropping removed useful context (96% hummus fell to 92% malawach)	rejected; centre-ROI kept
13	Edge executable failed on a clean machine	PyInstaller build was missing a hidden dependency (scikit-learn)	dependency pinned into the build spec; install verified on a second PC
14	ONNX edge probabilities drifted from PyTorch	preprocessing mismatch - cv2 resize order and normalisation differed from Ultralytics' transform	torch-free preprocess rewritten to replicate the exact transform; parity re-verified to 0% top-1 mismatch
15	Field-test hummus scored 0/3	**presentation-domain shift**: plastic takeaway container vs restaurant-plated training data	diagnosed (not hidden); fix defined as data **diversity** across presentations, not volume
16	`label_smoothing=0.1` in the training script had no effect	audit of the run's `args.yaml` + the Ultralytics classify trainer showed the argument is **silently ignored** (plain cross-entropy)	claim corrected in this book (Section 3.3.4 note); calibration re-attributed to dropout, augmentation and batch noise

The register reads as a sequence of setbacks; the point is that **none of them ended the project**. Each failure was measured, understood, and converted into either a fix or a documented design decision - the OCR failure produced the BLE channel, the forgetting failure produced the ensemble, the overconfidence failure produced the open-set class, and the leakage failure produced the evaluation discipline that makes every other number in this book credible.

9.4. Summary of Achievements

- A dual-model ensemble with a learned XGBoost router achieving **86.2%** system top-1 across 145 food classes, above the 80% target.
- A Global classifier at **88.18%** top-1 on verified leakage-free splits.
- A router lifted to **ROC-AUC 0.973** by adding an open-set "background" signal, raising Israeli recall from 61% to 79%, with the gain coming from information rather than threshold tuning.
- A deterministic BLE weight channel, recovered by reverse engineering, replacing two rejected vision-based methods.
- A torch-free **ONNX edge build** verified to 0% top-1 mismatch and running an analysis in 0.35 s on CPU.
- An end-to-end desktop application that fuses class and weight into a USDA-backed nutrition report, validated on real photographs.

9.5. Future Work and Improvements

The error analysis points the way. Because the ceiling is the models, the highest-value work is **more and cleaner data** for the weak classes (hummus and the small ingredient classes were the clear field-test failures), and optionally a larger backbone now that the 8 GB VRAM budget has headroom. The architecture also invites **more experts**: the router design adds a new cuisine as a new specialist without touching the others, so an Asian or Indian expert is a natural extension. On the product side, the desktop prototype could move to a mobile or fully embedded target using the existing ONNX build, and the single-item assumption could be relaxed with detection and segmentation to handle mixed plates.

10. References

The works below are cited at the point of use as numbered footnotes and consolidated here by theme. arXiv identifiers, DOIs and URLs link directly to the primary source.

10.1. Architectures, Classification and Transfer Learning

1. J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," **CVPR**, 2016. arXiv:1506.02640.
2. G. Jocher and J. Qiu, **Ultralytics YOLO11**, 2024. github.com/ultralytics/ultralytics
3. J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A Large-Scale Hierarchical Image Database," **CVPR**, 2009. image-net.org

10.2. Optimisation and Regularisation

1. I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization" (AdamW), **ICLR**, 2019. arXiv:1711.05101.
2. I. Loshchilov and F. Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts," **ICLR**, 2017. arXiv:1608.03983.
3. N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," **JMLR**, vol. 15, pp. 1929-1958, 2014. jmlr.org.

10.3. Ensemble Learning, Boosting and Interpretability

1. T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," **KDD**, 2016. arXiv:1603.02754.
2. S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions" (SHAP), **NeurIPS**, 2017. arXiv:1705.07874.

10.4. Out-of-Distribution Detection and Open-Set Recognition

1. D. Hendrycks and K. Gimpel, "A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks" (MSP), **ICLR**, 2017. arXiv:1610.02136.
2. D. Hendrycks, M. Mazeika, and T. G. Dietterich, "Deep Anomaly Detection with Outlier Exposure," **ICLR**, 2019. arXiv:1812.04606.
3. M. Hein, M. Andriushchenko, and J. Bitterwolf, "Why ReLU Networks Yield High-Confidence Predictions Far Away from the Training Data," **CVPR**, 2019. arXiv:1812.05720.

10.5. Continual Learning and Catastrophic Forgetting

1. J. Kirkpatrick et al., "Overcoming catastrophic forgetting in neural networks" (EWC), **PNAS**, vol. 114, no. 13, pp. 3521-3526, 2017. arXiv:1612.00796.

10.6. Edge / Browser Inference and Efficiency

1. S. Williams, A. Waterman, and D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," **Communications of the ACM**, vol. 52, no. 4, pp. 65-76, 2009. doi:10.1145/1498765.1498785.
2. ONNX and ONNX Runtime. onnxruntime.ai

10.7. Data Integrity, Metrology and Evaluation

1. S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in Data Mining: Formulation, Detection, and Avoidance," **ACM TKDD**, vol. 6, no. 4, art. 15, 2012. doi:10.1145/2382577.2382579.
2. E. B. Wilson, "Probable Inference, the Law of Succession, and Statistical Inference" (Wilson score interval), **JASA**, vol. 22, pp. 209-212, 1927. doi:10.1080/01621459.1927.10502953.
3. R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, "Towards Robust Monocular Depth Estimation" (MiDaS), **IEEE TPAMI**, 2020. arXiv:1907.01341. (Evaluated and rejected for weight estimation; see Section 8.)
4. A. Meyers et al., "Im2Calories: Towards an Automated Mobile Vision Food Diary," **ICCV**, 2015. (Prior RGB-D food-diary work; reports portion size as the dominant error - the same wall CalEyeZ avoids.)

10.8. Data Sources, Tools and Project Documents

1. U.S. Department of Agriculture, **FoodData Central**. fdc.nal.usda.gov
2. H. Blidh et al., **Bleak** - a cross-platform BLE library for Python. github.com/hbldh/bleak
3. CalEyeZ project site and living technical report (this project), Raz Dvora and Roi Tzur, Shenkar. raz-dv-ee.github.io/CalEyeZ
4. CalEyeZ FRS and SDD documents (internal, 2025-2026).

11. Appendices

11.1. List of Acronyms

Acronym	Meaning
AMP	Automatic Mixed Precision (training)
BLE	Bluetooth Low Energy
CNN	Convolutional Neural Network
CPU / GPU	Central / Graphics Processing Unit
GATT	Generic Attribute Profile (BLE)
GUI / HMI	Graphical User Interface / Human-Machine Interface
OCR	Optical Character Recognition
ONNX	Open Neural Network Exchange
ROC-AUC	Area Under the Receiver Operating Characteristic curve
ROI	Region of Interest
SHAP	SHapley Additive exPlanations
USDA	United States Department of Agriculture
VRAM	Video Random-Access Memory
V&V	Verification and Validation
YOLO	You Only Look Once (object-detection / classification family)

11.2. Software Code Snippets

The complete, runnable source lives in the project repository (github.com/raz-dv-ee/CalEyeZ); the key algorithms are reproduced in Sections 3 and 5. The principal scripts are:

Script	Role
`scripts/training/train_general_model.py`	Trains the 132-class Global classifier
`scripts/training/train_israeli_food_v2.py`	Trains the Israeli specialist (with background class)
`scripts/arbiter/generate_arbiter_dataset.py`	Builds the router feature table from both models
`scripts/arbiter/train_arbiter_xgb.py`	Trains and evaluates the XGBoost router
`scripts/edge/onnx_export_and_check.py`	Exports both models to ONNX and verifies parity
`scripts/ble/scale_reader.py`	Decodes the BLE scale and reports stable weight
`scripts/demo/caleyez_demo.py`	The full fusion application (GUI)

11.3. User Manual (Quick Start)

1. Connect the webcam and power on the SWAN BLE scale; launch the application. The app scans for and connects to the scale automatically.
2. Optionally set a USDA API key (environment variable or key file beside the executable) to enable online nutrition lookup; without it the local database is used.
3. Place a single food item on the scale and wait for the weight reading to read STABLE.
4. Click **Analyze**. The system shows the recognised food, the routing decision, and the calorie and macronutrient breakdown for the measured weight.
5. Use the camera-switch control if the machine has more than one camera.

11.4. Testing Procedures Tables

The quantitative results produced by the test plan of Section 5 are tabulated in Section 8: the per-model accuracy table, the router V1-vs-V2 table, the system-vs-baseline-vs-oracle table, the ONNX parity figures, the field-test per-class table, and the tried-and-rejected comparison table. They are not duplicated here.